Computers & Security 171 (2026) 105125

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

What You See Is Not What You Execute: Memory-Based Runtime SBOM Generation for Supply Chain Security

Hala Ali [a],*, Andrew Case [b], Irfan Ahmed [a]

[a] Department of Computer Science, Virginia Commonwealth University, USA
[b] Volatility Foundation, USA

ARTICLE INFO

Dataset link: https://github.com/HalaAli198/MEM-SBOM

Keywords:
Supply chain security
SBOM
Memory forensics
Volatility 3

ABSTRACT

Modern software development relies heavily on third-party components from public repositories, expanding the software supply chain attack surface. In response to these growing risks, federal initiatives have advanced the Software Bill of Materials (SBOM) as a standardized mechanism for improving transparency by describing software components, dependencies, and their relationships. However, SBOMs built from metadata or filesystem artifacts fail to capture the components loaded and executed at runtime, especially in dynamic ecosystems such as Python. Moreover, generating runtime SBOMs through instrumentation requires monitoring to be deployed in advance and the system to remain observable throughout execution. Such conditions are difficult to satisfy in production environments and incident-response scenarios. Volatile memory, in contrast, provides a reliable source for recovering the actual runtime state of a running application without requiring prior instrumentation. Therefore, this paper presents MEM-SBOM, the first memory forensics framework that generates SBOMs directly from the runtime state of Python applications. It recovers the modules from the interpreter's internal structures, resolves package versions, and analyzes bytecode to build dependency graphs and identify reachable vulnerable functions. We implemented MEM-SBOM as a suite of Volatility 3 plugins and evaluated it against 51 real-world Python applications. It achieves 100% extraction accuracy under non-adversarial conditions, identifies *Streamlit* as the only application that calls the vulnerable routines of the *tornado* dependency, and recovers all runtime packages missed by existing SBOM tools, providing more accurate dependency graphs and better vulnerability assessment. These capabilities make MEM-SBOM a practical foundation for software supply chain security and incident response by providing a forensically sound runtime view of what is executed on a system.

1. Introduction

The widespread adoption of third-party libraries and frameworks from public repositories has become integral to modern software development, enabling rapid feature integration while simultaneously expanding the software supply chain attack surface (Nahum et al., 2024; Cybersecurity and Infrastructure Security Agency (CISA), 2021; National Counterintelligence and Security Center (NCSC), 2021). Adversaries are increasingly exploiting these repositories to distribute malicious packages that compromise downstream applications (Vu et al., 2023). In 2024, more than 700,000 malicious packages were discovered in public registries, a 156% increase over the previous year (Sonatype, 2025). The official Python repository, *PyPI*, has repeatedly faced such incidents, with more than 12,000 malicious packages removed in 2022 alone (Fiedler, 2023), yet malicious uploads persist despite repository-level scanning (Samaana et al., 2025; Kampourakis et al., 2025). Examples include `zlibxjson v8.2`, which embedded credential-stealing scripts that exfiltrated cookies and authentication tokens (Wang, 2024), and `fshec2`, which concealed credential theft and remote command handling within compiled bytecode (`.pyc`) to evade detection (Nelson, 2023). In April 2025, additional malicious packages (`bitcoinlibdbfix`, `bitcoinlib-dev`, and `disgrasya`) were downloaded more than 39,000 times before being removed, with the latter embedding an automated script for credit-card fraud (Lakshmanan, 2025). Even trusted frameworks can introduce critical vulnerabilities, such as the recent SQL-injection flaw in *Django* (CVE-2025-64459), with affected versions still publicly available on PyPI (Bidart, 2025) at the time of writing.

These recurring incidents have highlighted the urgent need for software supply chain transparency, prompting regulatory and industry efforts to improve visibility into software composition. In response, initiatives led by the Cybersecurity and Infrastructure Security Agency

* Corresponding author.
E-mail addresses: alih16@vcu.edu (H. Ali), andrew@dfir.org (A. Case), iahmed3@vcu.edu (I. Ahmed).

https://doi.org/10.1016/j.cose.2026.105125
Received 19 November 2025; Received in revised form 13 July 2026; Accepted 18 August 2026
Available online 25 August 2026

(CISA) and the National Telecommunications and Information Administration (NTIA) have advanced the Software Bill of Materials (SBOM) as a standardized mechanism for software transparency and risk management (Office of the Director of National Intelligence (ODNI) et al., 2023). An SBOM is a structured inventory of software components and dependencies, specifying their metadata (e.g., name, version, and license) and relationships (Tooling and Implementation Working Group, 2024). Based on CISA's guidance, SBOMs have become an increasingly expected requirement of federal software procurement (Cybersecurity and Infrastructure Security Agency (CISA), 2025). As a result, several tools have emerged to automate SBOM generation across ecosystems. However, these tools remain constrained by their reliance on static metadata, such as build manifests and package files (e.g., `requirements.txt`, `pyproject.toml`, and `setup.py`), which describe what was built rather than what actually executes. Prior studies show that these metadata-driven tools generate incomplete and inaccurate SBOMs due to inconsistent format support, parser divergence, and weak handling of dynamic or environment-specific dependencies (Yu et al., 2024; Dalia et al., 2024; Cofano et al., 2024; Xia et al., 2023; Stalnaker et al., 2024). Furthermore, metadata files themselves can be compromised, as attackers may modify dependency specifications to mislead SBOM tools into producing incorrect component and version information (Zahan et al., 2023).

Recent efforts have extended SBOM generation to the software deployment phase, including container-level analysis (Kawaguchi and Hart, 2024), package-manager integration (Benedetti et al., 2025), and integrity verification of deployed components (Sharma et al., 2024). Although these approaches improve accuracy compared to static build-phase methods, they still rely on filesystem artifacts and package managers, which reflect the software's state at install. This limitation becomes even more evident in dynamic, interpreted languages, such as Python, where dependencies are resolved during execution rather than at build time. Runtime SBOMs address this limitation by capturing the active components of running applications through system instrumentation (Mirakhorli et al., 2024). However, this approach assumes the system can be safely monitored throughout execution. In production environments, such monitoring introduces operational overhead and risks perturbing the runtime state under investigation (Beyer et al., 2016; Reichelt et al., 2024). Moreover, in incident-response scenarios, compromised systems cannot be safely paused, debugged, or retroactively instrumented. Existing runtime SBOM tools reflect these constraints, as they require the target system to be live, observable, and instrumented prior to the incident under investigation, including agent-based platforms such as Rezilion's Dynamic SBOM (Rezilion, 2022), eBPF-based monitors such as Oligo Security (Oligo Security, 2023), and instrumentation-API tools such as Eclipse jbom (Eclipse Foundation, 2022). Beyond deployment constraints, based on publicly available documentation, these approaches observe application behavior externally without analyzing the internal state of loaded components, constructing runtime dependency graphs, or tracing vulnerable code paths within them, while still deriving package versions from installation metadata.

Volatile memory addresses these gaps by providing a reliable source for recovering the actual runtime components and dependencies directly from language runtimes, without requiring prior instrumentation or a live system. In this work, we target Python, given the need for language-specific SBOM tooling (Stalnaker et al., 2024), and the inconsistent results produced by cross-language tools across ecosystems (Halbritter and Merli, 2024; Mirakhorli et al., 2024). Python poses unique challenges for SBOM generation due to its highly flexible import mechanism that allows modules to be loaded at runtime through conditional, delayed, or reflective imports (Ali et al., 2025b), causing different frameworks and applications to load and retain dependencies in distinct ways. For example, when upgrading shared dependencies such as *asgiref*, *Django*'s *StatReloader* dynamically loads the newer version into memory during execution, whereas *Celery* lacks an auto-reload mechanism and continues using the older version already loaded at startup.

Therefore, in this paper, we propose MEM-SBOM, the first memory forensics framework that generates SBOMs directly from the runtime state of Python applications. It traverses the interpreter's internal structures, including module registries, thread contexts, the garbage collector, arenas, and heap regions, to extract all modules resident in memory. This multi-layered architecture ensures completeness and forensic soundness even in the presence of evasion techniques, such as module deletion and overwrite, import-hook interception, import-bypassing module creation, sub-interpreter injection, and garbage collector manipulation. Modules recovered from all layers and application processes are then aggregated, filtered to exclude built-in and standard-library components, and grouped by their top-level packages. To determine the versions of the extracted third-party packages, MEM-SBOM inspects version attributes across all modules, correlates them with installation metadata, and queries PyPI when necessary before serializing these packages into a CycloneDX-compliant SBOM. Beyond generating SBOMs, MEM-SBOM analyzes the Python objects associated with each package and disassembles the bytecode of its functions to construct dependency graphs that capture both direct and transitive runtime relationships. Leveraging the resulting SBOMs and graphs and integrating public vulnerability data, the framework performs fine-grained, function-level reachability analysis to distinguish the exploitable code paths at runtime.

We implemented MEM-SBOM as a suite of new Volatility 3 plugins (Volatility Foundation, 2025). We then evaluated it across 51 real-world Python applications spanning 23 diverse categories, including CLI tools, web frameworks, machine learning platforms, workflow schedulers, and visualization tools. Validated against a dual static and runtime ground truth, MEM-SBOM accurately extracts the metadata cached for installed packages and recovers all application packages, including those dynamically loaded at runtime, across heterogeneous execution environments. It further identifies ten cases where the version embedded in the loaded code differs from the version declared at installation time. In a *tornado* case study, function-level reachability analysis shows that only *Streamlit*, among six *tornado*-dependent applications, invokes the vulnerable routines at runtime, eliminating 83.3% of false-positive vulnerability reports. Finally, our comparison against eight state-of-the-art SBOM tools shows that none of them support all Python metadata configurations or capture dynamically loaded packages, resulting in incomplete SBOMs, partially constructed dependency graphs, and ultimately inaccurate vulnerability reports.

Our contributions are as follows:

- We propose MEM-SBOM, the first framework to generate SBOMs directly from the runtime state of Python applications, providing an execution-grounded view of software components.
- We characterize evasion techniques that conceal Python modules during execution and design a robust multi-layered extraction architecture capable of detecting and recovering all modules.
- We construct runtime dependency graphs and perform function-level vulnerability reachability analysis, identifying exploitable code paths.
- We evaluate MEM-SBOM across 51 real-world Python applications, achieving higher accuracy and vulnerability precision than existing SBOM tools.

The remainder of this paper is organized as follows. Section 2 provides the background, and Section 3 summarizes related work. Section 4 discusses the threat model. Section 5 outlines the key challenges, while Section 6 explains the methodology. Section 7 presents the evaluation, Section 8 discusses generalizability, limitations, and future directions, and Section 9 concludes the paper.

2. Background

This section presents SBOM types and standards, followed by an overview of the import mechanism and memory analysis of the Python runtime.

2.1. SBOM fundamentals

SBOMs can be generated at different stages of the software lifecycle, each providing a distinct level of visibility into software composition (Cybersecurity and Infrastructure Security Agency (CISA), 2023). **Build SBOMs** describe the components, dependencies, and build metadata incorporated during compilation or packaging. **Deployed SBOMs** enumerate the components installed and configured within operational environments. **Runtime SBOMs**, in turn, capture the components actively loaded during execution, revealing dynamic and environment-specific dependencies that are invisible to static analysis. SBOMs are commonly represented using two standards. **SPDX**, maintained by the Linux Foundation, focuses on license compliance, copyright attribution, and file-level provenance (Software Package Data Exchange (SPDX), 2025). **CycloneDX**, developed by OWASP, is a lightweight, security-oriented standard that emphasizes dependency relationships, component provenance, and vulnerability correlation (CycloneDX Project, 2025b).

2.2. Import mechanism of python

The import mechanism of Python follows a multi-stage resolution process formalized in *PEP 451* and *PEP 420* (Snow, 2013; Smith, 2012). When the interpreter encounters an `import` statement, it first checks its per-interpreter registry (`sys.modules`), which maps module names to loaded `PyModuleObject` instances. If the module is not found, the import machinery proceeds through finder objects listed in `sys.meta_path`. Each finder implements `find_spec()` to locate the requested module and, upon success, returns a `ModuleSpec` that encapsulates the module's name, origin, loader, and other import metadata. Using this specification, the interpreter calls `loader.create_module(spec)` if provided, or creates a new module object otherwise, registers it in `sys.modules`, and executes it via `loader.exec_module(module)` to populate its namespace. Built-in and frozen modules use non-filesystem origins (e.g., "built-in" or "frozen"), whereas file-based modules populate attributes, such as `__file__` and `__cached__`. This spec-driven model replaces the legacy `load_module()` API, separating module discovery (`find_spec()`) from execution (`exec_module()`) and exposing import state through the `__spec__` attribute for reliable runtime introspection.

2.3. Memory analysis of python

Python's runtime architecture consists of interdependent memory management subsystems that control object allocation and persistence in memory. The global `_PyRuntime` structure, of type `_PyRuntimeState`, represents the entry point to the runtime, maintaining interpreter registries, thread states, and the configuration of the garbage collector. This structure is located in the dynamic symbol table of *ELF* binaries on Linux or the export table of *PE* binaries on Windows (Ali et al., 2025a). Each interpreter instance (`PyInterpreterState`) maintains its own registry, import machinery, and a set of thread contexts (`PyThreadState`), where each thread holds a stack of execution frames containing references to active function objects (Ali et al., 2025b). Memory allocation follows a hybrid scheme. Small objects (≤512 bytes) are managed by the `pymalloc` allocator within 256 KB arenas subdivided into 4 KB pools (Golubin, 2019). Larger objects are handled by the system allocator via `brk()` for the process heap and `mmap()` for separate anonymous memory mappings, depending on allocation size and system configuration. The garbage collector tracks objects across three generations using doubly-linked lists via `gc_prev` and `gc_next` pointers. These mechanisms define the spatial organization of Python objects in memory, enabling the reconstruction of complete runtime SBOMs from memory artifacts.

3. Related work

Evaluating Existing SBOM Tools. Prior studies primarily evaluated SBOM tools, standardization challenges, and adoption barriers. Dalia et al. (2024) evaluated *Syft*, *Microsoft SBOM Tool*, *Tern*, and *CycloneDX* tools in terms of multi-platform support, format compatibility, and usability, demonstrating that these tools remain static, metadata-driven, and lacking operational readiness. Mirakhorli et al. (2024) analyzed 84 tools and observed heavy reliance on package managers, inconsistent component identification, and limited cross-language coverage, urging the need for standardized dependency semantics and validation frameworks. Halbritter and Merli (2024) further showed that even the most accurate tools (*CycloneDX-Npm* and *CycloneDX-Conan*) fail to meet the *NTIA*'s minimum-element requirements, frequently omitting dependencies or misreporting versions. From an adoption perspective, Xia et al. (2023) found that standardization issues, inadequate validation, and information-sensitivity concerns hinder adoption. Stalnaker et al. (2024) identified specification complexity, tooling insufficiency, and verification issues as major barriers, and highlighted the need for language-specific and verifiable SBOM frameworks, particularly for emerging AI/ML domains lacking formal standards. Bi et al. (2024) analyzed 4786 SBOM-related GitHub discussions, linking poor SBOM quality to technical debt and governance deficiencies, highlighting the need for continuous assurance integration. Complementary studies by Cofano et al. (2024) and Yu et al. (2024) highlighted consistent reliability issues in *Trivy*, *Syft*, *CDXGen*, *ORT*, *Microsoft SBOM Tool*, and *GitHub Dependency Graph*, attributing inaccuracies to flawed metadata parsing, ambiguous naming and version-resolution, and incomplete dependency resolution, reflecting the inherent limitations of static, metadata-driven analysis.

Proposing New SBOM Techniques. Recent efforts have extended SBOM generation to the deployment phase of software. Sharma et al. (2024) introduce *SBOM.EXE*, which prevents dynamic code injection in Java by maintaining a Bill of Materials Index (BOMI) of legitimate class checksums. This approach is inherently limited since it requires complete test coverage and cannot handle non-deterministic code generation or hidden classes. Benedetti et al. (2025) propose *PIP-SBOM*, which integrates SBOM creation into the Python package manager *pip* using its native resolver. However, this approach reports versions that would be freshly installed rather than actual deployed versions, and inherits pip's limitations in failing to capture runtime-only dependencies and dynamic imports. Kawaguchi and Hart (2024) developed *C3DRS*, a consumer-side vulnerability management system that generates SBOMs through dynamic container inspection. Although it improves detection accuracy, its analysis remains limited to filesystem artifacts.

Runtime SBOM Approaches. A separate class of tools generates SBOMs by instrumenting running applications. Rezilion's Dynamic SBOM (Rezilion, 2022) continuously tracks software components across hosts, containers, and cloud environments, determining which are loaded and assessing their exploitability. Oligo Security (2023) uses eBPF-based kernel-level probes to trace syscalls and monitor library behavior at function granularity, distinguishing between loaded and executed dependencies. Eclipse jbom (Eclipse Foundation, 2022) takes a language-specific approach for Java, using the JVM's `Instrumentation` API via ByteBuddy agent attachment to enumerate all loaded classes in running JVMs and produce CycloneDX SBOMs. Although these tools enhance runtime visibility, they share two fundamental limitations. First, they require the target system to remain live and observable during analysis. Second, based on their publicly available documentation, these tools observe application behavior externally and derive package versions from installation metadata, without analyzing loaded bytecode to construct dependency graphs. Regarding vulnerability assessment, Rezilion and Oligo report which components or functions executed during monitoring rather than analyzing bytecode to enumerate reachable paths to vulnerable code, while Eclipse jbom does not provide vulnerability assessment capabilities. MEM-SBOM

addresses these limitations. It goes beyond component inventory by recovering modules from multiple sources and aggregating them into a single application-level SBOM. It resolves package versions from the loaded code itself and performs bytecode-level analysis to construct dependency graphs and trace reachability paths to vulnerable functions.

4. Threat model

Adversary goal. The adversary aims to distribute malicious Python packages through public repositories such as PyPI and GitHub, inject them into an application's dependency chain, and ensure they execute on the victim's system without being detected by platform scanners, SBOM generation tools, or runtime analysis.

Adversary capabilities. The adversary can craft malicious packages and distribute them by impersonating legitimate packages through typosquatting, starjacking, and dependency confusion, or by compromising existing packages through account takeover and repository hijacking. Once the adversary controls a package, it can manipulate the package's metadata, including version information and dependency names, to mislead SBOM generation tools and exploit inconsistencies between package managers and SBOM tool parsers (Yu et al., 2024). The adversary can further exploit Python's flexibility to evade static detection by distributing malicious payloads as compiled bytecode (`.pyc`) or through conditional and deferred imports that are triggered only at runtime. At the runtime level, the adversary can conceal injected modules by deleting or overwriting them after injection, intercepting imports through malicious hooks, creating modules without registering them, injecting modules into sub-interpreters, or unlinking them from the garbage collector.

Assumptions. The Python runtime structures are assumed to be intact; the adversary targets data managed by the runtime, not the runtime's own integrity. Malicious modules operate within the privileges of the Python process. Memory acquisition faithfully preserves process state at the time of capture. The adversary is further assumed to have knowledge of detection strategies employed by memory forensics tools. Adversarial techniques that fall outside the current detection surface include type-pointer spoofing, payloads stored outside Python's module object model, and code executing exclusively through native extensions. These require extending detection beyond module-level analysis and are discussed in Section 8.2.

5. Challenges

This section identifies six challenges that MEM-SBOM is designed to overcome.

C1: Evasive manipulation of the Python import system. The dynamic nature of Python allows introspection and modification of its import system during execution. The same flexibility that enables extensibility, via monkey patching, custom import hooks, or dynamic loaders, can be exploited to conceal or alter modules in memory. Attackers may delete or overwrite entries in the interpreter registry (`sys.modules`), replace legitimate modules with malicious substitutes, create modules that never appear in `sys.modules` using the `importlib` and `runpy` APIs, inject hidden modules into sub-interpreters, or unlink module objects from the garbage collector by manipulating their pointers directly through `ctypes`. Such behaviors complicate the recovery of a complete and trustworthy module set from memory.

C2: Fragmented runtime state in multi-process applications. Complex Python applications (e.g., *Airflow*, *FastAPI*, *MLflow*) comprise multiple cooperating processes, each maintaining its own interpreter state and dependency set. These processes may load different or conflicting versions of shared packages depending on the deployment environment. To generate an accurate, application-level SBOM, these fragmented module views must be merged while resolving version conflicts and cross-process duplications.

C3: Distinguishing third-party packages from internal and standard libraries. When a Python application runs, the interpreter instantiates not only the application's modules but also a wide range of built-in and standard-library modules required for its own operations. In memory, all of these appear as `PyModuleObject` instances within the same runtime, making it difficult to isolate third-party components of the application. Differentiating these modules is essential for constructing an accurate SBOM that reflects only third-party dependencies.

C4: Incomplete version information in memory. Accurately resolving package versions from memory poses a major challenge for memory-based SBOM generation. A `PyModuleObject` may expose version-related attributes, such as `__version__`, `VERSION`, `__VERSION__`, `version_info`, `__version_info__`. However, these attributes lack standardization, exhibit inconsistent naming, and are not guaranteed to be present in memory. This challenge complicates the generation of accurate memory-based SBOMs, for which package name and version are essential elements.

C5: Structural over-approximation of dependency relationships. Python's eager import mechanism loads entire namespaces into memory, making it difficult to distinguish between modules that are merely loaded and those actually used. When a module imports another, the callee's modules, classes, and functions are instantiated in memory, even if most remain unreferenced. Recursive imports further amplify this effect by pulling additional transitive dependencies into memory. Traversing these imports results in deep transitive chains that populate memory with modules the application never invokes, leading to inflated SBOMs with high false-positive rates. This, in turn, causes the dependency graph to over-approximate the true runtime state, reporting relationships that are syntactically derived but semantically invalid. Therefore, with such structural over-approximation, constructing precise runtime SBOMs poses another challenge.

C6: Identifying exploitable code paths. Beyond C5's module-level over-approximation, this challenge extends to the internal objects of each module. Once imported, all of a module's objects, such as functions, classes, and attributes, reside in memory, even though many of them may never be used by the application. Thus, the presence of a vulnerable function in memory does not imply that it is reachable from the application. Determining which portions of a module are affected by a vulnerability requires inspecting all of its objects and enumerating the possible call chains across all application modules, which becomes a core technical challenge.

6. Methodology

Fig. 1 illustrates the MEM-SBOM framework. After identifying the application processes and locating the interpreter's internal structures within the memory dump (Ali et al., 2025b), the framework extracts the runtime modules to produce a CycloneDX-compliant SBOM, a dependency graph, and function-level exploitation paths. These three outputs are connected through a progressive pipeline. The SBOM enumerates the application's runtime packages and their versions, which are then scanned against public vulnerability databases to identify affected components and their vulnerable functions. The dependency graph, constructed through bytecode analysis of import and call relationships, captures how packages depend on each other, constraining the scope of subsequent analysis. The reachability analysis then traces backward along these dependencies at the function level, identifying which application components have execution paths that reach the vulnerable code.

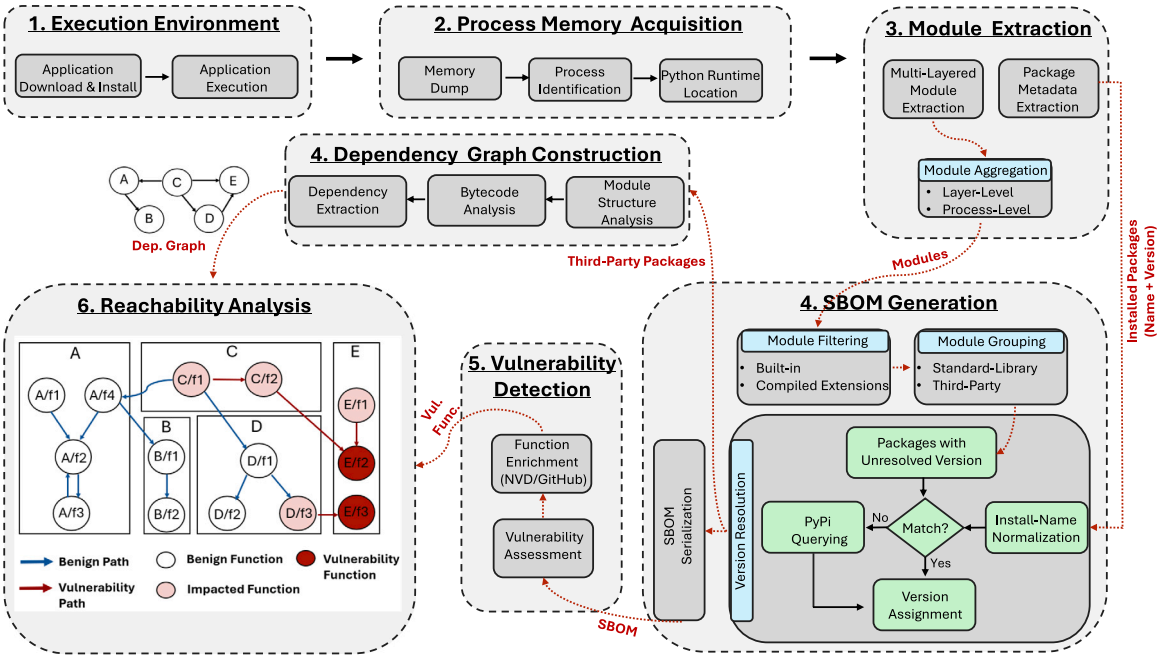


Fig. 1. Overview of the MEM-SBOM Framework.

The remainder of this section follows this pipeline. Section 6.1 describes module extraction, Section 6.2 covers SBOM generation, Section 6.3 constructs the dependency graph, and Sections 6.4 and 6.5 present vulnerability detection and reachability analysis.

6.1. Module extraction

6.1.1. Package metadata extraction

To identify installed packages without filesystem access, MEM-SBOM inspects the Python import system cache (i.e., `sys.path_importer_cache`), which maps filesystem paths to finder objects responsible for module discovery (see Section 2.2). Each `FileFinder` instance maintains a persistent `_path_cache` containing all recognized directory entries, including source files (`.py`), bytecode files (`.pyc`), extension modules (`.so`, `.pyd`), and package-metadata directories (`.dist-info`, `.egg-info`). This cache is constructed once at import time, providing a stable record of the interpreter's package environment. MEM-SBOM extracts the package metadata (i.e., name and version) from cached directory entries that match the *PEP 427* wheel format (`package_name-version.dist-info`) or legacy `egg-info` naming scheme. It then normalizes the recovered names, associates version numbers, and identifies their installation scope based on the directory paths.

6.1.2. Multi-layered module extraction

Python's runtime structures can be exploited to conceal malicious modules through various evasion techniques, as discussed next.

A1. Module Deletion. Deletes entries from `sys.modules`, hiding modules that remain live through residual references elsewhere in memory.

A2. Module Overwrite. Overwrites or aliases legitimate entries in `sys.modules` (e.g., mapping a benign name to a malicious module object) to hide the module behind a trusted identifier.

A3. Import-Hook Interception. Inserts a malicious finder into `sys.meta_path` that redirects a targeted module import to a custom loader, which executes a malicious module during initialization.

A4. Import-Bypassing Module Creation. Creates modules via `importlib` and `runpy` APIs without registering them in `sys.modules`.

A5. Sub-Interpreter Injection. Injects malicious modules into secondary interpreters (sub-interpreters) within the same process, making them invisible to scans of the main interpreter's registry.

A6. Garbage-Collector (GC) Manipulation. Manipulates the pointers of GC-tracked objects (e.g., via `ctypes`) to unlink malicious modules from its doubly-linked lists, allowing them to evade detection.

To address these evasion techniques, MEM-SBOM employs a robust multi-layered extraction approach that overcomes Challenge **C1** by incrementally expanding coverage from interpreter registries to raw heap regions. Layers 1–3 provide authoritative and context-rich evidence with minimal false positives, ideal for benign module recovery. Layers 4–5 extend extraction to raw memory arenas and heap regions. Although these layers require object-type validation, they are essential for achieving completeness, as all modules with active references remain allocated in memory.

L1. Interpreter Registry. Traverses `sys.modules` of each interpreter, providing the fastest extraction mechanism.

L2. Thread Execution States. Inspects the global namespace and local variables of the stack frames to identify their modules that might be hidden from the interpreter registry.

L3. Garbage Collector Traversal. Traverses the doubly-linked lists of objects managed by the process-level garbage collector, which is shared across all interpreters.

L4. Arena-Aware Analysis. Scans arena pools to identify valid module objects within allocated blocks by verifying their reference count (`ob_refcnt`) and type (`ob_type`).

L5. Heap Areas Scanning. Recovers large module objects in 8-byte aligned increments, interpreting 16-byte sequences as `ob_refcnt` and `ob_type`, validating the `PyObject` header, dereferencing the type pointer if it corresponds to a module type, and confirming module identity.

Module Aggregation. To address Challenge **C2**, MEM-SBOM implements two levels of aggregation. First, within each application process, it deduplicates modules discovered across all five extraction layers by name. Second, for multi-process applications, per-process module sets are merged to generate the application's complete module set.

6.2. SBOM generation

After module extraction, MEM-SBOM retains only the application packages for version resolution, excluding built-in and standard-library components from the SBOM. These packages are then serialized into a CycloneDX-compliant SBOM for further analysis.

6.2.1. Module filtering

A Python process includes the application's own modules, standard-library modules, and a set of built-in modules that are compiled directly into the interpreter. The application itself is structured as a main package with its dependent packages, each composed of one or more `.py` files represented in memory as `PyModuleObject` instances (Snow, 2013; Smith, 2012). The main package corresponds to the distribution's top-level package (e.g., *streamlit*, *Django*, *Pelican*) while all other imported packages are treated as dependencies. Addressing Challenge **C3**, MEM-SBOM first filters out the built-in modules (e.g., `_signal`, `marshal`, and `_imp`) that are identified through the interpreter's `sys.builtin_module_names` registry. It further excludes the compiled extension modules (e.g., `.so`, `.pyd`) that represent native C extensions. The remaining modules (i.e., standard-library and third-party) are retained for grouping and top-level package mapping in the next stage.

6.2.2. Module grouping

Each module is identified by its fully qualified name (FQN), which specifies its position within the package hierarchy. For example, the package `requests` with its modules (`adapters`, `models`, and `sessions`) are represented in memory as `requests`, `requests.adapters`, `requests.models`, and `requests.sessions`. MEM-SBOM groups modules by their top-level package, defined as the substring preceding the first dot (`.`) in the module name. This grouping recovers a one-to-many relationship between a package and the modules that compose it, enabling accurate package-level representation in the resulting SBOM. Among these top-level packages, the application's main package appears as one of them and is grouped in the same way as all other packages. After grouping, the framework classifies each package as standard-library or third-party based on its installation path and excludes the standard-library packages. Third-party packages are identified by paths such as `site-packages`, `dist-packages`, or environment-specific vendor directories (e.g., `vendor-packages`, `local-packages`). Standard-library packages are recognized through name-based heuristics and path patterns corresponding to the interpreter's installation directories (e.g., `/usr/lib/python3.x/`, `/lib/python3.x/`).

Some packaging tools bundle their own dependencies to ensure version consistency, creating namespace-isolated copies of third-party packages. For instance, *pip* embeds its dependencies under `pip._vendor` (e.g., `pip._vendor.requests`), while *setuptools* uses `setuptools._vendor`. MEM-SBOM identifies and excludes these bundled copies by matching common namespace conventions (e.g., `._vendor.`, `.extern.`) and directory patterns (e.g., `/_vendor/`, `/extern/`). Internal utility packages, such as `_distutils_hack`, are similarly omitted to ensure the SBOM reflects only the application packages.

6.2.3. Version resolution

MEM-SBOM resolves the versions of the packages identified in the previous stage through a three-stage fallback process.

Step 1: Version attribute inspection. MEM-SBOM analyzes the `PyModuleObject` structures to inspect their version attributes (e.g., `__version__`, `VERSION`, `version_info`). As noted in Challenge **C4**, these attributes are not mandatory and therefore might not appear in memory.

Step 2: Local cache matching. When version fields cannot be located, MEM-SBOM attempts to map each import name to its installation record in `sys.path_importer_cache` to retrieve the associated version (see Section 6.1.1). However, distribution names in installation records differ from their import counterparts due to the normalization rules defined in *PEP 503* (Stufft, 2015). For example, the distribution `email-validator` corresponds to the import name `email_validator`. Such discrepancies are resolved by applying *PEP 503* normalization (i.e., case folding and hyphen-to-underscore transformation) before matching import and installation names.

Step 3: PyPI resolution. A more complex case arises when normalization fails to establish a correspondence between the import names observed in memory and the installation names in the local cache (`sys.path_importer_cache`). For instance, `sys.modules` may contain `cv2`, while the cache lists `opencv-contrib-python`, and `opencv-python-headless`. Although these distributions originate from the same upstream *OpenCV* project, they install distinct packages that all expose the same import name (`cv2`), making it impossible to infer from the local cache alone which distribution provided the loaded module. To resolve such ambiguity, and given that the PyPI API operates on distribution identifiers rather than import names, MEM-SBOM queries PyPI using the previously unmatched installation names discovered in the local cache. The framework retrieves package metadata through the public JSON API, extracting import names from the `.dist-info/top_level.txt` file for wheel distributions and from the `.egg-info/top_level.txt` file for source distributions. This file enumerates the top-level importable names defined by each distribution.

This process allows the recovery of the complete set of import names corresponding to the locally installed distributions. MEM-SBOM then matches these recovered import names against the modules observed in memory to determine their originating distributions. Once a match is found, the version information is obtained from the local cache, ensuring that the generated SBOM reflects the actual environment state. Ultimately, when the version attribute remains indeterminate, MEM-SBOM records its value as `''unknown''`, consistent with CISA's SBOM minimum-element specification (Cybersecurity and Infrastructure Security Agency (CISA), 2025).

6.2.4. SBOM serialization

After resolving package versions, MEM-SBOM serializes the package information into a CycloneDX-compliant SBOM in JSON format. We selected this standard due to its security-oriented design and support for vulnerability correlation, which aligns with the cybersecurity and digital forensics support objectives of our work. Following CISA's guidance (Tooling and Implementation Working Group, 2024), we focus on attributes essential to incident response (e.g., component identity, version, and dependency relationships) rather than licensing or copyright metadata. The generated SBOM consists of four primary sections. **Metadata.** Defines SBOM provenance using a UUID-based serial number, extraction timestamp, and tool identifier to ensure extraction traceability. **Components.** Lists each package with fields `name`, `version`, and package URL (PURL) following (`pkg:pypi/package@version`) format, enabling direct mapping to vulnerability databases such as National Vulnerability Database (NVD), Open Source Vulnerabilities (OSV), and GitHub Security Advisories (GHSA). Component hashes are omitted because memory-resident modules lack stable file artifacts, consistent with CISA's allowance for inaccessible sources. Additional forensic metadata, such as `extracted_from: memory` to distinguish runtime from manifest-based SBOMs, and `file_path` for filesystem correlation, are preserved in the `properties` of the component. **Dependencies.** Lists the direct dependencies of the application packages. **Annotations.** Includes the annotator identity, creation timestamp, extraction source, and package count, allowing analysts to assess completeness and maintain provenance across multiple SBOMs.

6.3. Dependency graph construction

This section addresses Challenge **C5** by describing how MEM-SBOM constructs the dependency graph among the recovered application packages. The construction combines two complementary sources of evidence. Namespace analysis (Section 6.3.1) identifies the external objects each module references, while bytecode analysis (Section 6.3.2) recovers import statements embedded in function bodies, including those executed conditionally or dynamically. Section 6.3.3 integrates both sources into a package-level graph.

6.3.1. Module structure analysis

The namespace dictionary (`__dict__`) of each `PyModuleObject` contains references to the module's own objects as well as those imported from other modules. For each object in the dictionary, MEM-SBOM determines its origin based on its type:

Module objects are attributed through the `md_name` attribute.

Class objects are attributed through the `__module__` attribute, with recursive inspection of the class dictionary to capture nested dependencies.

Callable objects (e.g., functions, generators, coroutines, asynchronous generators, instance methods, class methods, and static methods) are attributed through the `func_module` attribute, with recursive inspection of inner functions.

6.3.2. Bytecode analysis

To extract import statements embedded within the function body, we disassemble its compiled bytecode and apply a stride-1 sliding window of width four over the opcode sequence to capture the complete instruction patterns associated with import variants. During traversal, non-semantic scaffolding instructions (e.g., `RESUME`, prologue assignments, and constant initializations) are excluded to reduce noise. Each window is then compared against the opcode patterns summarized in Table 1, which covers standard imports (P1–P3), from-imports (P4–P8), and dynamic imports via `exec()`, `importlib`, and `__import__()` (P9–P11).

For patterns P1–P8, the module name is extracted directly from the `IMPORT_NAME` operand, while imported symbols are ignored since they do not affect package-level relationships. For dynamic imports (P9–P11), the target name is recovered from the `LOAD_CONST` string operand, with `exec()` strings additionally parsed via regular expressions to extract embedded import statements.

Note. The bytecode analysis maintains cross-version compatibility by handling instruction-set variations across Python releases. Bound-method calls compiled as `LOAD_METHOD` → `CALL_METHOD` in Python 3.6–3.10 are replaced in Python 3.11 by `LOAD_ATTR`→ `CALL`, optionally followed by `KW_NAMES` to handle keyword arguments. For name resolution, module-level code uses `LOAD_NAME`→ `STORE_NAME`, while function scopes use `LOAD_GLOBAL`→ `STORE_GLOBAL` for global variables, `LOAD_FAST` → `STORE_FAST` for local variables, and `LOAD_DEREF` → `STORE_DEREF` for closure variables. When `LOAD_CONST` instructions have a code object as their argument value, the analysis recursively inspects them to recover imports from all inner scopes.

6.3.3. Dependency extraction

Algorithm 1 constructs the dependency graph by integrating namespace analysis and bytecode inspection. For each application package $p \in \mathcal{P}$, let $\mathcal{M}_p$ denote the set of modules belonging to p. For each module $m \in \mathcal{M}_p$, the algorithm initializes an empty import set $\mathcal{I}_m$ and traverses the module's dictionary. For each object o in the dictionary, it determines the object type τ through the `ob_type` field. When τ is `module` and the referenced module name belongs to a different package than p, it is identified as an external dependency and appended to $\mathcal{I}_m$. For class objects, it extracts the `__module__` attribute and recursively processes the class dictionary to capture nested dependencies. For callable objects, it extracts the `func_module` reference, validates it as an external dependency, analyzes its bytecode, and merges the results into $\mathcal{I}_m$. After computing $\mathcal{I}_m$ for all modules of package p, the algorithm aggregates and deduplicates imports across $\mathcal{M}_p$, and creates a directed edge $E(p, q)$ for each distinct imported package q, yielding a package-level dependency graph.

Algorithm 1 ***Dependency Graph Construction***

Input: $\mathcal{P}$ (application packages)
Output: $\mathcal{D}_{graph}$ (dependency graph)
Initialize $\mathcal{D}_{graph} \leftarrow \emptyset$
for each package $p \in \mathcal{P}$ **do**
  Initialize $\mathcal{I}_p \leftarrow \emptyset$ {Imports observed for package p}
  for each module $m \in \mathcal{M}_p$ **do**
    Initialize $\mathcal{I}_m \leftarrow \emptyset$
    ProcessDict($p, m.\_dict\_, \mathcal{I}_m$)
    $\mathcal{I}_p \leftarrow \mathcal{I}_p \cup \mathcal{I}_m$
  end for
  for each imported package $q \in \mathcal{I}_p$ **do**
    $\mathcal{D}_{graph} \leftarrow \mathcal{D}_{graph} \cup \{E(p, q)\}$
  end for
end for

Procedure ProcessDict($p, dict, \mathcal{I}_m$)
for each object $o \in dict$ **do**
  $\tau \leftarrow o$.ob_type.tp_name
  if τ = module **then**
    $mod\_name \leftarrow o.\_name\_$
    if $pkg(mod\_name) \neq p$ **then**
      $\mathcal{I}_m \leftarrow \mathcal{I}_m \cup \{mod\_name\}$
    end if
  else if τ = class **then**
    $class\_mod \leftarrow o.\_module\_$
    if $pkg(class\_mod) \neq p$ **then**
      $\mathcal{I}_m \leftarrow \mathcal{I}_m \cup \{class\_mod\}$
    end if
    ProcessDict($p, o.\_dict\_, \mathcal{I}_m$) {Recurse into class}
  else if $\tau \in Callable\_Types$ **then**
    $func\_mod \leftarrow o$.func_module
    if $pkg(func\_mod) \neq p$ **then**
      $\mathcal{I}_m \leftarrow \mathcal{I}_m \cup \{func\_mod\}$
    end if
    $\mathcal{I}_{bytecode} \leftarrow$ BytecodeAnalysis(o.func_code)
    $\mathcal{I}_m \leftarrow \mathcal{I}_m \cup \mathcal{I}_{bytecode}$
  end if
end for

Fig. 2 illustrates how the resulting dependency graph serves as the foundation for vulnerability analysis. Once the vulnerability scanner identifies an affected package and function enrichment determines its vulnerable modules and functions, the graph is used to identify which packages have direct or transitive edges leading to the vulnerable target. This narrows the analysis scope from the entire application to only the packages along the relevant paths. Without this graph, identifying the impact of a vulnerability would require inspecting every function across all application packages. The reachability analysis builds on the same bytecode disassembly technique introduced for dependency graph construction. While the dependency graph extracts import patterns (Table 1) to identify package-level relationships, the reachability analysis extends this technique to extract function call patterns (Table 2), enabling fine-grained traversal of execution paths within the narrowed scope. The following section describes how the framework leverages this narrowed scope to perform function-level reachability analysis.

Table 1
Bytecode patterns corresponding to Python import constructs.

Pattern	Python code example	Bytecode sequence pattern

P1: Direct import
P2: Multiple direct imports
P3: Direct import with alias
	`import math`
	`import os, sys`
	`import math as m`
		`IMPORT_NAME(math)` → `STORE_FAST(math)` / `STORE_NAME(math)`
		Repeat: (`IMPORT_NAME` → `STORE_FAST` / `STORE_NAME`) for each module
		`IMPORT_NAME(math)` → `STORE_FAST(m)`/ `STORE_NAME(m)`

P4: From import (Single module)	`from math import sqrt`	`IMPORT_NAME(math)` → `IMPORT_FROM(sqrt)` → `STORE_FAST(sqrt)`/ `STORE_NAME(sqrt)` → `POP_TOP`
P5: From import (Multiple modules)	`from math import sqrt, pi, cos`	`IMPORT_NAME(math)` → Repeat: (`IMPORT_FROM` → `STORE_FAST` / `STORE_NAME`) for each item → `POP_TOP`
P6: From import with alias	`from math import sqrt as sq`	`IMPORT_NAME(math)` → `IMPORT_FROM(sqrt)` → `STORE_FAST(sq)` / `STORE_NAME(sq)` → `POP_TOP`
P7: From import star	`from math import *`	`IMPORT_NAME(math)` → `IMPORT_STAR`
P8: From import (Submodules)	`from os.path import join`	`IMPORT_NAME(os.path)` → `IMPORT_FROM(join)` → `STORE_FAST(join)` / `STORE_NAME(join)` → `POP_TOP`
P9: Dynamic import via `exec()`	`exec('import math')`	`LOAD_GLOBAL/LOAD_NAME(exec)` → `LOAD_CONST(string)` → `CALL_FUNCTION/CALL`
P10: Dynamic import via `importlib`	`import importlib m = importlib.import_module('math')`	`IMPORT_NAME(importlib)` → … → `STORE_FAST(importlib)` / `STORE_NAME(importlib)` → `LOAD_METHOD/LOAD_ATTR(import_module)` → `LOAD_CONST('math')` → `CALL_METHOD/CALL`
P11: Dynamic import via `__import__()`	`m = __import__('math')`	`LOAD_GLOBAL/LOAD_NAME(__import__)` → `LOAD_CONST('math')` → `CALL_FUNCTION/CALL`

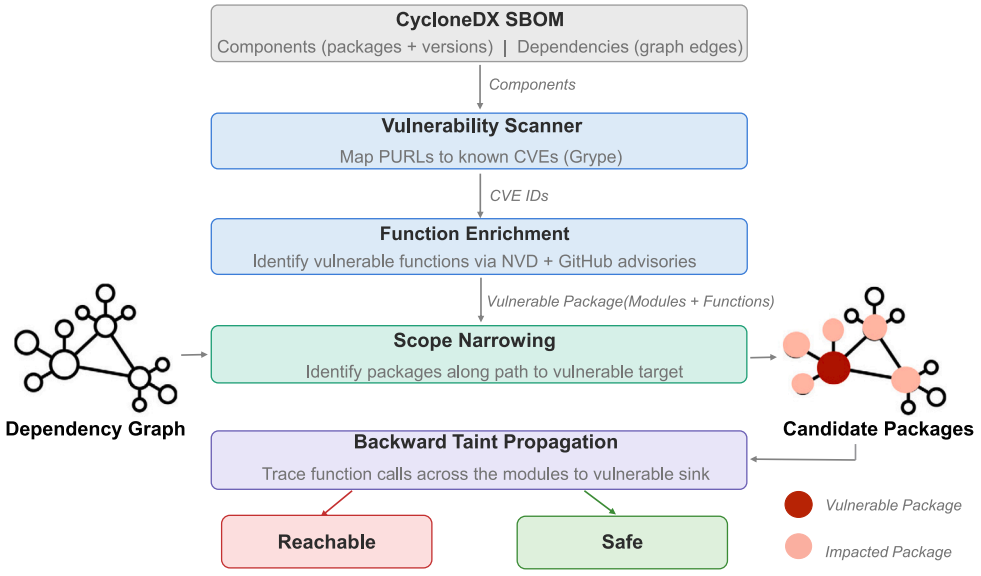


Fig. 2. Workflow illustrating the correlation between the SBOM, dependency graph, and vulnerability reachability analysis.

6.4. Vulnerability detection

To begin addressing Challenge **C6**, MEM-SBOM integrates the SBOM, dependency graph, and public vulnerability data in two steps, package-level vulnerability identification (this section) followed by function-level enrichment (Section 6.4.1), to identify vulnerable packages within the application, determine their impact scope, and trace all reachable paths to them. To identify vulnerabilities, we use *Grype* (v0.100.0) as the vulnerability scanner due to its accuracy, ecosystem coverage, and competitive performance compared to alternative scanners (Anchore, 2025a; Benedetti et al., 2025; Bufalino et al., 2025). This tool maps SBOM components (via their PURLs) to known CVEs using aggregated advisory feeds (e.g., NVD, OSV, and GitHub Security Advisories). For each SBOM component, it produces a structured set of matching vulnerabilities, including identifiers, affected version ranges, and severities, which MEM-SBOM attaches to the corresponding packages as input to the function-level analysis.

6.4.1. Function-level enrichment

Given the over-approximation inherent in package-level vulnerability detection, as discussed in Challenges **C5** and **C6**, MEM-SBOM augments CVE matches with function-level metadata extracted from public vulnerability advisories. This enrichment enables us to distinguish vulnerable code paths within the application. For each CVE reported by *Grype*, we query NVD and GitHub Security Advisories to obtain additional context. From NVD, we retain entries whose CPE configurations match the affected package and apply heuristics over the free-text CVE description to extract function identifiers. From GitHub advisories, when remediation commits or pull requests are available, we analyze the corresponding diffs to identify updated function definitions. When function-level attribution cannot be reliably determined, either due to incomplete advisory descriptions or genuinely package-wide impact, we label the CVE with package-wide scope and leave finer-grained localization to manual analysis.

Table 2
Bytecode patterns corresponding to Python function call constructs.

Pattern	Python code example	Bytecode sequence pattern

F1: Inner function call	`def outer():`
	`  def inner(): ...`
	`  inner()`
		Definition: `LOAD_CONST(inner code object)` → `MAKE_FUNCTION` → `STORE_FAST(inner)`
		Call: `LOAD_FAST(inner)` → `CALL_FUNCTION` / `CALL` / `CALL_FUNCTION_KW` / `CALL_FUNCTION_EX`

F2: Closure function call	`def outer():`
	`  x = 10`
	`  def inner():`
	`  return x`
	`  inner()`
		Definition: `LOAD_CLOSURE(x)` → `LOAD_CONST(inner code object)` → `MAKE_FUNCTION(closure)` → `STORE_FAST(inner)`
		Call: `LOAD_FAST(inner)` → `CALL_FUNCTION` / `CALL` / `CALL_FUNCTION_KW` / `CALL_FUNCTION_EX`

F3: Closure/Nonlocal call	`def outer():`
	`  def helper(): ...`
	`  def inner():`
	`  helper()`
		`LOAD_DEREF(helper)` → `CALL_FUNCTION` / `CALL` / `CALL_FUNCTION_KW` / `CALL_FUNCTION_EX`

F4: Decorator wrapper pattern call	`def wrapper(*args, **kwargs):` `  return func(*args, **kwargs)`	**Definition:** `LOAD_CONST(inner code object)` → `MAKE_FUNCTION(closure)` → `STORE_FAST(wrapper)` **Inside wrapper - calling original:** `LOAD_DEREF(func)` → `LOAD_FAST(args)` → `LOAD_FAST(kwargs)` → `CALL_FUNCTION_EX`
F5: Function call (instance/class/module)	`obj = MyClass(), obj.method()` `module.func()`	`LOAD_METHOD()` / `LOAD_ATTR()` → ... → `CALL_METHOD`/ `CALL` / `CALL_FUNCTION_KW` / `CALL_FUNCTION_EX`

6.5. Reachability analysis

While Section 6.4 identifies which packages contain vulnerable functions, this section determines whether the application can actually execute them, completing Challenge **C6**. MEM-SBOM determines which parts of the application are exposed to identified CVEs by tracing exploitation paths through backward taint propagation. Given a vulnerability sink $S = (p, m, f)$, where p denotes the vulnerable package, m the module, and f the function, it traces backward from f to identify all callers that eventually reach the application's main package. This produces taint propagation paths $\langle f_1, f_2, \ldots, f_n \rangle$, where $f_n = f$ is the vulnerable function and f_1 belongs to the application's main package. The sink is considered reachable if at least one such path exists.

The analysis proceeds in two stages. First, within the vulnerable package, we inspect all modules, classes, and functions to identify local callers of f. Any function that invokes f, either directly or through an internal call chain within the same package, is added to the sink set. Second, for each package that imports the vulnerable package, we detect functions that directly call any element of the current sink set and mark them as vulnerable. We then apply the same intra-package analysis to these caller packages to identify internal call chains that reach those functions. The resulting set of direct and internal callers forms the sink set for the next iteration. This recursive expansion continues until the process reaches the application's main package. At that point, we perform the same intra-package analysis on the main package to enumerate all application functions involved in the call chain. A vulnerability is considered reachable if any backward traversal originating in the main package reaches the vulnerable function.

This process requires analyzing function bytecode to extract call relationships. Therefore, we extend the bytecode analysis introduced in Section 6.3.2 to detect the call patterns shown in Table 2, including inner-function calls, closure and nonlocal calls, decorator-wrapper patterns, and instance, class, and module-level function calls.

7. Evaluation

This section first validates the capabilities of MEM-SBOM in generating accurate runtime SBOMs across heterogeneous installation environments, constructing dependency relationships, and enabling precise vulnerability impact assessment. It also demonstrates its superiority over existing state-of-the-art tools through a set of experiments in terms of runtime package recovery, dependency-graph completeness, and vulnerability correlation.

7.1. Experimental setup

We implemented MEM-SBOM as a suite of Volatility 3 (v2.26.2) plugins running on Ubuntu 20.04.6 LTS with 8 GB of RAM and Python 3.8.10. Application processes were identified using Volatility's `pslist` and `pstree` plugins. The reachability analysis plugin takes as input the application's process ID, main package (source), vulnerable package (target), vulnerable function, and the dependency graph, returning all runtime paths leading to the vulnerable code as output. The evaluated applications use Python versions ranging from 3.6 to 3.14. While the memory architecture remains stable across these versions, performance-oriented refinements introduce variations in internal offsets (e.g., the addresses of `_PyRuntime` and `arenas`). MEM-SBOM considers these version-specific offsets to maintain compatibility across all supported Python versions.

7.1.1. Application deployment

We installed each of the applications using `pip install` into an isolated Python virtual environment created on Ubuntu 20.04 LTS virtual machines with 4 vCPUs and 4 GB RAM. This isolation ensures that each application is evaluated with its own dependency set, preventing any cross-application interference and guaranteeing that all modules recovered by MEM-SBOM belong solely to the target application. Memory dumps were captured by snapshotting the virtual machine immediately after the application completed its initialization phase, allowing sufficient time for it to complete startup and reach a steady execution state in which all the packages used by the application were resident in memory. Modules related to environment management and packaging utilities (e.g., `pip`, `pkg_resources`, `setuptools`, `wheel`, `distlib`, and `virtualenv`) as well as application-specific configuration files were excluded from comparisons to ensure fair comparison between the MEM-SBOM and SBOM tools.

7.1.2. Dataset

We evaluated 51 open-source Python applications from the *Awesome-Python* repository (Chen, 2025). Each selected application constitutes a standalone system with its own dependency tree, metadata configuration, and runtime context, thereby capturing the diversity of Python packaging and execution models observed in practice. The dataset covers a wide range of 23 categories, including CLI tools (e.g., *iRedis*, *LiteCLI*), web frameworks (e.g., *Django*, *Flask*, *FastAPI*), machine learning platforms (e.g., *MLflow*, *BentoML*, *Rasa*), task schedulers (e.g., *Apache Airflow*, *Celery*, *Prefect*), and data visualization tools

(e.g., *Apache Superset*, *Orange*). This diversity reflects the heterogeneity of real-world Python deployments and enables evaluating MEM-SBOM under realistic operational conditions. The complete dataset is presented in Table 14 of Appendix.

Note. The versions installed for each application and its dependencies are determined by the underlying operating system and environment configuration. Consequently, the versions resolved by `pip` may not correspond to the most recent releases available on PyPI. Our dataset, therefore, reflects realistic deployment environments rather than enforcing the latest releases.

7.1.3. Ground-truth generation

To evaluate the correctness of MEM-SBOM in recovering both installed package metadata and packages actively used at runtime, we constructed two complementary forms of ground truth: a static ground truth and a runtime ground truth. The static ground truth captures the set of packages installed in each application environment. We obtained this information using `pip freeze`, which enumerates all installed distributions and their resolved versions. This dataset serves as the reference for validating MEM-SBOM's ability to reconstruct package names and versions directly from memory without filesystem access.

The runtime ground truth reflects the set of modules loaded by the Python interpreter during application execution. It is derived from the interpreter registry (`sys.modules`), which provides the authoritative record of legitimately imported modules in benign, non-adversarial deployments (see Section 6.1.2). Each application is instrumented using Python's `sitecustomize` module, a built-in startup mechanism that the interpreter automatically imports before any application-level code (Python Software Foundation, 2024), ensuring that our monitoring logic executes at interpreter initialization. The instrumentation captures the complete contents of `sys.modules` after the application reaches a steady execution state. It also reinitializes automatically in child processes spawned at runtime, providing complete coverage for multi-process applications. When triggered, each process serializes its modules into a process-specific JSON file. Memory snapshots were acquired immediately thereafter, ensuring temporal alignment between the live module state and its in-memory representation. The same processing steps described in Sections 6.2.1 and 6.2.2 (i.e., module filtering and module grouping) are applied to the runtime ground-truth data to isolate the application modules. This dual-ground-truth strategy provides complementary visibility into installed distributions and actively used packages, enabling accurate assessment of MEM-SBOM's module-extraction accuracy. After validating MEM-SBOM against both views, we used it as the authoritative runtime baseline to evaluate the SBOM tools, while the static ground truth continues to serve as the reference for installed-package accuracy comparisons.

7.1.4. SBOM tools

We evaluated widely used SBOM tools: *Syft* (v1.27.1) (Anchore, 2025b), *Trivy* (v0.65.0) (Aqua Security, 2025), *CDXGen* (v11.4.4) (CycloneDX Project, 2025a), *CycloneDX-Python* (v7.1.0) (CycloneDX Project, 2025c), *SBOM4Python* (v0.12.4) (Harrison, 2025), *ORT* (v68.2.0) (OSS Review Toolkit, 2025), *Jake* (v3.0.14) (Sonatype Nexus Community, 2025), and *PIP-SBOM* (Benedetti et al., 2025). For metadata-based evaluation, all eight tools were assessed. For deployment-based evaluation, we restricted the comparison to *Syft*, *CDXGen*, *CycloneDX-Python*, and *PIP-SBOM*, as these tools can analyze an existing installed environment or construct a synthetic one for SBOM generation. *Syft* and *CycloneDX-Python* directly inspect the evaluated environment by enumerating installed distributions from the `site-packages` directory. *CDXGen*, in contrast, simulates an installation environment by creating a temporary virtual environment, resolving and installing all dependencies declared in the application's manifests, and enumerating the resulting `site-packages` directory. *PIP-SBOM* does not inspect installed environments; instead, it downloads the application's distribution into a temporary directory, extracts its metadata, and resolves dependencies using PIP's native logic. As a result, it reports the versions PIP would install, not the exact versions present in the evaluated environment.

7.2. Evaluation research questions

Our evaluation addresses the following research questions:

RQ1: How accurately can MEM-SBOM recover the runtime state of Python applications, including installed, imported, and dynamically loaded packages?

RQ2: Can MEM-SBOM determine the actual reachability of known vulnerabilities within running applications?

RQ3: How do existing SBOM generation tools perform across diverse metadata configurations and real deployment environments, and how do their SBOMs impact the vulnerability detection accuracy compared to MEM-SBOM?

7.2.1. Evaluation metrics

We evaluate MEM-SBOM and existing SBOM tools using four complementary categories of metrics that capture the package identification, dependency graph construction, vulnerability detection accuracy, as well as vulnerability impact and reachability. All metrics described in this section are computed per application, and their mean values are calculated across the full evaluation dataset to summarize the aggregate performance of the tools and MEM-SBOM.

Package identification. Let P_S, P_T, and P_M denote the sets of packages obtained from the static ground truth (`pip freeze`), the evaluated SBOM tool, and MEM-SBOM, respectively.

We compute:

$$C_P = \frac{|P_T \cap P_M|}{|P_M|} \times 100\%, \quad P_P = \frac{|P_T \cap P_M|}{|P_T|} \times 100\%,$$

$$F_P = 2 \cdot \frac{C_P \cdot P_P}{C_P + P_P}, \quad M_P = \frac{|(P_S \cap P_M) \setminus P_T|}{|P_M|} \times 100\%,$$

$$R_P = \frac{|P_M \setminus (P_S \cup P_T)|}{|P_M|} \times 100\%$$

C_P (*Coverage* or *Recall*) measures the proportion of runtime packages correctly detected by the tool. P_P (*Precision*) measures the proportion of tool-reported packages that are confirmed at runtime. F_P (*F1 Score*) provides the harmonic mean of precision and recall, capturing the balance between completeness and correctness. M_P (*Missing*) quantifies packages that exist in both static and runtime environments but are missing from the tool output, and R_P (*Runtime-Only*) captures runtime-only dependencies invisible to static analysis.

Version accuracy. For each package present in all of the static ground truth (P_S), the tool output (P_T), and the runtime set recovered by MEM-SBOM (P_M), we evaluate whether the version reported by the tool ($v_T(p)$) or by MEM-SBOM ($v_M(p)$) matches the static ground-truth version ($v_S(p)$). Version accuracy is the proportion of such matched versions and is defined as follows:

$$V_{acc} = \frac{|\{\, p \in (P_S \cap P_T \cap P_M) \mid v_X(p) = v_S(p) \,\}|}{|P_S \cap P_T \cap P_M|} \times 100\% \quad X \in \{T, M\}$$

Dependency graph construction. For each package p, let $D_T(p)$ and $D_M(p)$ denote its direct dependencies in the graphs generated by the evaluated SBOM tool and MEM-SBOM, respectively, and let E_T and E_M be the corresponding sets of directed edges. We define the common-edge domain (E_{comm}) as:

$$E_{\text{comm}} = \{\, (u \to v) \in E_M \;:\; u, v \in V_T \cap V_M \,\}$$

where V_T and V_M are the node sets of the tool and MEM-SBOM graphs, respectively. Using these definitions, we evaluate structural accuracy through three metrics defined as follows:

$$D_{acc} = \frac{\sum_{p \in P_M} |D_T(p) \cap D_M(p)|}{\sum_{p \in P_M} |D_M(p)|} \times 100\%, \qquad E_{acc} = \frac{|E_T \cap E_M|}{|E_{\text{comm}}|} \times 100\%,$$

$$G_{complete} = \frac{|E_T \cap E_M|}{|E_M|} \times 100\%$$

D_{acc} (*Dependency Accuracy*) measures how precisely a tool recovers direct runtime dependencies for packages observed in memory. E_{acc} (*Edge Accuracy*) measures how accurately the tool identifies runtime dependency edges for the packages that both the tool and MEM-SBOM have in common (E_{comm}), while $G_{complete}$ (*Graph Completeness*) evaluates global coverage by comparing all runtime edges in memory with those reconstructed by the tool, regardless of node presence.

Vulnerability detection. Let V_S, V_T, and V_M denote the sets of vulnerabilities reported by *Grype* when analyzing SBOMs generated using `pip-audit` (static ground truth), the evaluated tool, and MEM-SBOM, respectively. The static vulnerability baseline is obtained by executing `pip-audit` on the packages reported by `pip freeze`.

$$C_V = \frac{|V_T \cap V_M|}{|V_M|} \times 100\%, \quad P_V = \frac{|V_T \cap V_M|}{|V_T|} \times 100\%,$$

$$F_V = 2 \cdot \frac{C_V \cdot P_V}{C_V + P_V}, \quad M_V = \frac{|(V_S \cap V_M) \setminus V_T|}{|V_M|} \times 100\%,$$

$$R_V = \frac{|V_M \setminus (V_S \cup V_T)|}{|V_M|} \times 100\%.$$

C_V (*Coverage* or *Recall*) measures the proportion of runtime vulnerabilities (i.e., those identified by MEM-SBOM) that are correctly identified by the tool. P_V (*Precision*) measures the proportion of tool-reported vulnerabilities that are present at runtime. F_V (F1 Score) provides the harmonic mean of precision and recall, capturing the overall balance between detection completeness and correctness. M_V (*Missing*) measures vulnerabilities that appear in both the static ground truth and memory but are missing from the tool's output. Finally, R_V (*Runtime-Only*) represents the fraction of vulnerabilities associated with runtime-only packages.

Vulnerability impact and reachability. To measure how a vulnerable dependency propagates through the application's runtime graph, we employ three metrics. **DEC (Direct Exposure Count)** measures the number of packages that directly depend on the vulnerable package, indicating the extent of immediate exposure. **RP (Reachability Percentage)** quantifies the fraction of all packages in the runtime graph that have a direct or transitive path to the vulnerable package, capturing the scope of potential impact. **MDD (Minimum Dependency Distance)** reports the shortest path from the application's main package (source) to the vulnerable package (target), representing the depth of remediation required.

7.3. Evaluation results

This section presents and discusses the experimental results that answer the evaluation research questions.

7.3.1. RQ1: Package recovery

Table 3 summarizes the evaluation results of MEM-SBOM's extraction accuracy using 23 representative applications, each chosen to illustrate a distinct category from the complete set of 51 applications. The results show the exact matching between the package metadata recovered from memory by inspecting the `_path_cache` structure and the static ground truth obtained from `pip freeze`. This validates the ability of MEM-SBOM to accurately reconstruct the name and version of the packages installed in the deployment environment without relying on filesystem artifacts. Similarly, the comparison between the runtime ground truth and the modules recovered from `sys.modules` yields a perfect match, demonstrating the accuracy of MEM-SBOM in capturing the subset of installed packages that the application actually imports during execution. Measuring precision (fraction of recovered packages present in the ground truth) and recall (fraction of ground-truth packages recovered from memory), both yield 100% for installed and runtime package recovery across all 51 applications. As anticipated, not all installed packages are imported at runtime. This discrepancy arises from optional and conditional dependencies, platform-specific packages, and build-time utilities that are never used during execution. For instance, in system-wide installations, the Python environment includes numerous OS-bundled packages that are present on the filesystem but never imported by the application.

The results also illustrate a set of dynamic package modules that appear only at runtime. Such modules typically originate from packages dynamically loaded via `importlib.import_module()` or `__import__()`, side-loaded packages bundled directly within the application directory, or plugin frameworks that introduce additional modules during execution. *Glances* shows the largest runtime expansion (31 modules) due to its plugin architecture, which dynamically imports monitoring extensions from `glances/plugins/*.py` as independent top-level modules (e.g., `glances_cpu`, `glances_network`) via `importlib`, rather than hierarchical modules. Meanwhile, *Errbot*, *Apache Superset*, and *Modoboa* inject deployment-specific configurations through direct imports; *Datasette*, *Scapy*, and *Mayan-EDMS* expose backward-compatibility wrappers that create duplicated namespaces at runtime (e.g., `multipart/python_multipart`, `attr/attrs`).

Table 4 highlights the robustness of MEM-SBOM across six heterogeneous installation environments for the *Flask* web framework, representing diverse packaging ecosystems. Despite substantial variation in dependency resolution and metadata formats (e.g., `.dist-info` and `.egg-info`) across `pip`, `Conda`, `Poetry`, and system-level package managers, MEM-SBOM consistently enumerated the same eight imported packages of the application (i.e., `Flask`, `Itsdangerous`, `Jinja2`, `Blinker`, `click`, `Markupsafe`, `zipp`, and `Werkzeug`), demonstrating environment-independent extraction accuracy. However, the small differences in package counts reflect expected installation artifacts. Source-based installations introduce build tools (e.g., `wheel`) as a result of compiling the source archives using the package manager `pip`. *Conda* inherits 78 user-scope packages unless `PYTHONNOUSERSITE=1` is set. *Poetry* introduces a virtual infrastructure module (`_virtualenv`) for environment isolation while *APT*-based installations injects `apport_python_hook`, a system-level crash-reporting module. Notably, *APT*-based installations show the highest heterogeneity, combining 76 user-level and 70 system-wide packages across `site-packages` and `dist-packages`. When Flask is installed system-wide, its version and dependency set depend on the operating system's bundled Python packages. As a result, the application inherits legacy builds and distribution-specific variants (e.g., `Flask 1.1.1`) as distributed with Ubuntu 20.04, illustrating version divergence and mixed provenance that MEM-SBOM effectively exposes.

Beyond reconstructing an accurate runtime state from memory, MEM-SBOM identifies inconsistencies between the declared package version at installation and the version embedded in the loaded code. Table 5 highlights ten representative cases in our evaluation set where MEM-SBOM revealed such mismatches that metadata-based tools failed to detect. These inconsistencies arise for several reasons. First, version attributes embedded in the source code may remain unchanged across releases, leading to incorrect version values in memory. Second, development builds may contain prerelease identifiers that are not reflected in the package manifests. Third, wrapper or alias artifacts may expose imported module names that diverge from the identifiers of the underlying distribution packages.

The Ajenti-Panel case exemplifies the third category, where `beautifulsoup4==4.13.4` provides the `bs4` import namespace, while a separate compatibility package `bs4==0.0.2` is installed in the same environment. This results in two distribution packages mapping to a single runtime namespace. Such mismatches can conceal vulnerable versions or enable dependency confusion, misleading tools that rely solely on static metadata. By validating version information directly from memory, MEM-SBOM can prove whether the installed packages align with their declared provenance, a critical capability for generating accurate SBOMs.

Table 3
Accuracy of MEM-SBOM in recovering installed, imported, and dynamically loaded packages.

Application	Static packages		Imported packages		Dynamic
	Static ground truth	Memory (_path_cache)	Runtime ground truth	Memory (sys.modules)	modules
Ajenti-Panel	50	50	40	40	0
Apache Superset	136	136	102	102	2
APScheduler	10	10	9	9	0
Beets	21	21	20	20	1
Daphne	21	21	16	16	0
Datasette	27	27	21	21	1
DevPi-Server	66	66	45	45	0
Django	5	5	5	5	0
Errbot	33	33	30	30	0
Fsociety	14	14	12	12	0
Glances	5	5	35	35	31
iRedis	14	14	14	14	0
JupyterLab	91	91	69	69	0
Lektor	24	24	23	23	0
Locust	30	30	26	26	0
Mayan-EDMS	146	146	110	110	2
MLflow	64	64	29	29	0
Modoboa	86	86	60	60	2
RPyC	4	4	2	2	0
RQ	4	4	4	4	0
Scapy	1	1	1	1	0
Scrapy	37	37	28	28	1
Trytond	15	15	14	14	0

Table 4
Accuracy of MEM-SBOM in recovering the *Flask*'s packages across heterogeneous installation environments.

Install	Environment	Static packages		Imported	Flask
method	type	Env.	System	packages	version
pip (wheel, PyPI)	venv	12	0	8	3.0.3
pip (source, PyPI)	venv	13	0	8	3.0.3
pip (GitHub)	venv	12	0	8	3.0.3
Conda (user inherit)	Conda + user scope	0	78	8	3.0.3
Poetry	poetry venv	12	0	9	3.0.3
System (APT)	System-wide	0	146	9	1.1.1

Table 5
Version inconsistencies between package metadata and embedded code as detected by MEM-SBOM.

Application	Dependency	Installed version	Recovered version
Ajenti-Panel	beautifulsoup4	0.0.2	4.13.4
DevPi-Server	strictyaml	1.7.3	1.6.2
Jet-Bridge	regex	2024.11.6	2.5.148
Lektor	lazy_object_proxy	1.11.0	1.10.0
Mayan-EDMS	regex	2025.7.34	2.5.159
Modoboa	oath	1.4.4	1.4.0
MyCLI	pymysql	1.1.2	1.4.6
Prefect	regex	2025.9.1	2.5.161
Rasa	terminaltables	3.1.10	3.1.0
Spyder	textdistance	4.6.3	4.6.2

Scope and adversarial considerations. The 100% extraction accuracy reported above is achieved under non-adversarial conditions across 51 real-world applications, validated against dual ground truth. This result depends on the multi-layered architecture (L1–L5), in which each layer compensates for the blind spots of others. For example, modules deleted from `sys.modules` or unlinked from the garbage collector remain detectable through lower layers. To evaluate detection under adversarial conditions, we implemented the six evasion techniques (A1–A6) as separate installable Python packages. Each package's `__init__.py` executes at import time, injecting a hidden module using its technique. A driver script imports the package to trigger the injection, and a memory snapshot is acquired while the module remains resident. Table 6 reports the extraction layer at which the module is detected under each technique. We exclude the thread execution-state layer (L2), as the crafted modules define no functions that would be active on any thread's call stack. Scanning the interpreter registry misses the modules that are deleted after creation, overwrite benign existing modules, or are created without being registered, while the garbage collector misses the module that is unlinked from its linked lists. However, the deeper layers recover all hidden modules. The `PyModuleObject` is a fixed-size structure below pymalloc's 512-byte threshold, so it is allocated in an `obmalloc` arena pool regardless of its namespace size. The arena-aware pass thus recovers it by scanning these pools, and the heap scan by traversing all readable and writable regions.

However, the extraction fundamentally depends on the presence of a valid `PyModuleObject` with a type pointer resolving to `tp_name = ''module''`. Adversarial techniques that eliminate the `PyModuleObject` entirely (e.g., storing payloads in plain dictionaries or serialized byte buffers), spoof the `ob_type` pointer to a non-module type, or execute code exclusively through native extensions operate outside the detection surface of module-level enumeration. Addressing these adversarial scenarios requires extending detection beyond module enumeration to code-object-level analysis, bytecode pattern recognition, and cross-layer integrity validation, which we identify as directions for future work.

Answer to RQ1

Under non-adversarial conditions, MEM-SBOM achieves 100% coverage of installed, imported, and dynamically loaded packages across 51 real-world applications and heterogeneous deployment environments.

Table 6
Evaluation of MEM-SBOM against the six evasion techniques (A1–A6) across its extraction layers. ✓ indicates the module is detected by that layer; ✗ indicates it is not.

ID	Technique	Interpreter L1	Garbage collector L3	Arena L4	Heap L5
A1	Module Deletion	✗	✓	✓	✓
A2	Module Overwrite	✗	✓	✓	✓
A3	Import-Hook Interception	✓	✓	✓	✓
A4	Import-Bypassing Creation	✗	✓	✓	✓
A5	Sub-Interpreter Injection	✓	✓	✓	✓
A6	GC Manipulation	✗	✗	✓	✓

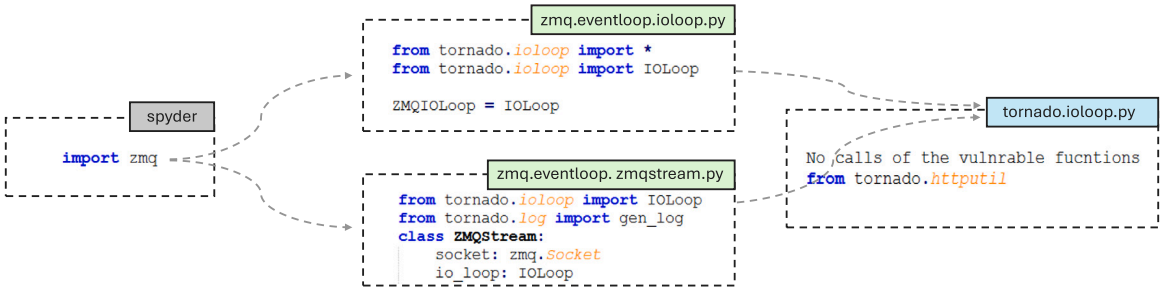


Fig. 3. Transitive dependency path from *spyder* to the vulnerable *tornado* package via *zmq*, showing that no vulnerable functions are reached.

7.3.2. RQ2: vulnerability reachability

Using *Grype*, MEM-SBOM identifies all known vulnerabilities present in the evaluated applications and their dependent packages, including *urllib3*, *tornado*, and *werkzeug* as illustrated in Table 15 of Appendix. As a representative case study, we focused on the *tornado* dependency, which is used by multiple applications and contains vulnerable routines located in the `tornado.httputil` module, namely `_unquote_cookie()`, `parse_cookie()`, `parse_body_arguments()`, and `parse_multipart_form_data()`. These functions are affected by *CVE-2024-52804* (unsafe HTTP header parsing) and *CVE-2025-47287* (unsafe multipart parsing). Despite the availability of patched releases, several applications continue to rely on outdated versions. We found that *BentoML*, *Flower*, *JupyterLab*, *Spyder*, and *Streamlit* embed *tornado* 6.4.2, while *Jet-Bridge* bundles *tornado* 5.1.1.

To illustrate how this vulnerable dependency propagates through each application, Table 7 reports the DEC (Direct Exposure Count), RP (Reachability Percentage), and MDD (Minimum Dependency Distance). All applications except *BentoML* and *Spyder* depend on *tornado* directly (MDD = 1). *JupyterLab* exhibits the widest reach, with eight of its packages linked to *tornado* (DEC = 8, RP = 14.3%), whereas smaller frameworks such as *Streamlit* and *Jet-Bridge* show limited propagation (RP = 2.9% and 5.9%, respectively). These variations illustrate how an application's architecture determines the extent to which a vulnerable dependency propagates through its components. However, the presence of a vulnerable package does not necessarily imply that its vulnerable functionality is reachable from the application.

Package-level scanning, therefore, overestimates risk. Fig. 3 illustrates that although `spyder` has a transitive dependency path to `tornado.ioloop` module via the `zmq` package, none of the vulnerable `httputil` functions are used. In contrast, Fig. 4 shows how `streamlit` reaches the vulnerable `parse_body_arguments()` function through its `UploadFileRequestHandler.put()` method, traversing the `web`, `server`, and `upload_file_request_handler` modules. The bytecode of `put()` confirms a direct call to the vulnerable function.

Table 7
Evaluation of vulnerability propagation for the *tornado* package across the evaluated applications.

Application	Version	#Pkgs	DEC	RP (%)	MDD	Dep. type
BentoML	1.3.5	54	2	7.4	**2**	Transitive
Flower	2.0.1	19	1	5.3	1	Direct
Jet-Bridge	1.12.1	35	1	2.9	1	Direct
JupyterLab	4.3.8	70	**8**	**14.3**	1	Direct
Spyder	6.0.8	105	3	6.7	**2**	Transitive
Streamlit	1.40.1	17	1	5.9	1	Direct

Table 8 summarizes the function-level reachability results. While all six applications are flagged as vulnerable at the package level, the detailed inspection reveals that five of them never call the affected routines. For instance, *Flower* imports several *tornado* modules (e.g., `web`, `ioloop`, `httpserver`), yet none originate from `tornado.httputil`. *JupyterLab* imports `httputil` but only uses benign routines such as `url_concat()` and `HTTPHeaders.parse_line()`, neither of which transitively invoke any vulnerable function. Only *Streamlit* demonstrates a verifiable exploitation path, making it the only application where the vulnerability is reachable in memory. Thus, while package-based scanners flag 6/6 applications (100%), function-level validation identifies only one true exposure (16.67%), eliminating 83.3% of false positives. This granularity shows that only *Streamlit*'s upload handler requires remediation rather than upgrading *tornado* across all dependent projects.

These results carry direct implications for vulnerability management in practice. In production environments, organizations maintain numerous applications with extensive dependency trees. When a new CVE is disclosed, package-level scanners report every application that includes the affected package as vulnerable, regardless of whether the affected routines are actually invoked at runtime. This over-reporting leads to inefficient allocation of security resources, as a significant proportion of such reports constitute false positives that require unnecessary investigation and remediation effort (Zhao et al., 2024). Function-level reachability analysis mitigates this by distinguishing applications

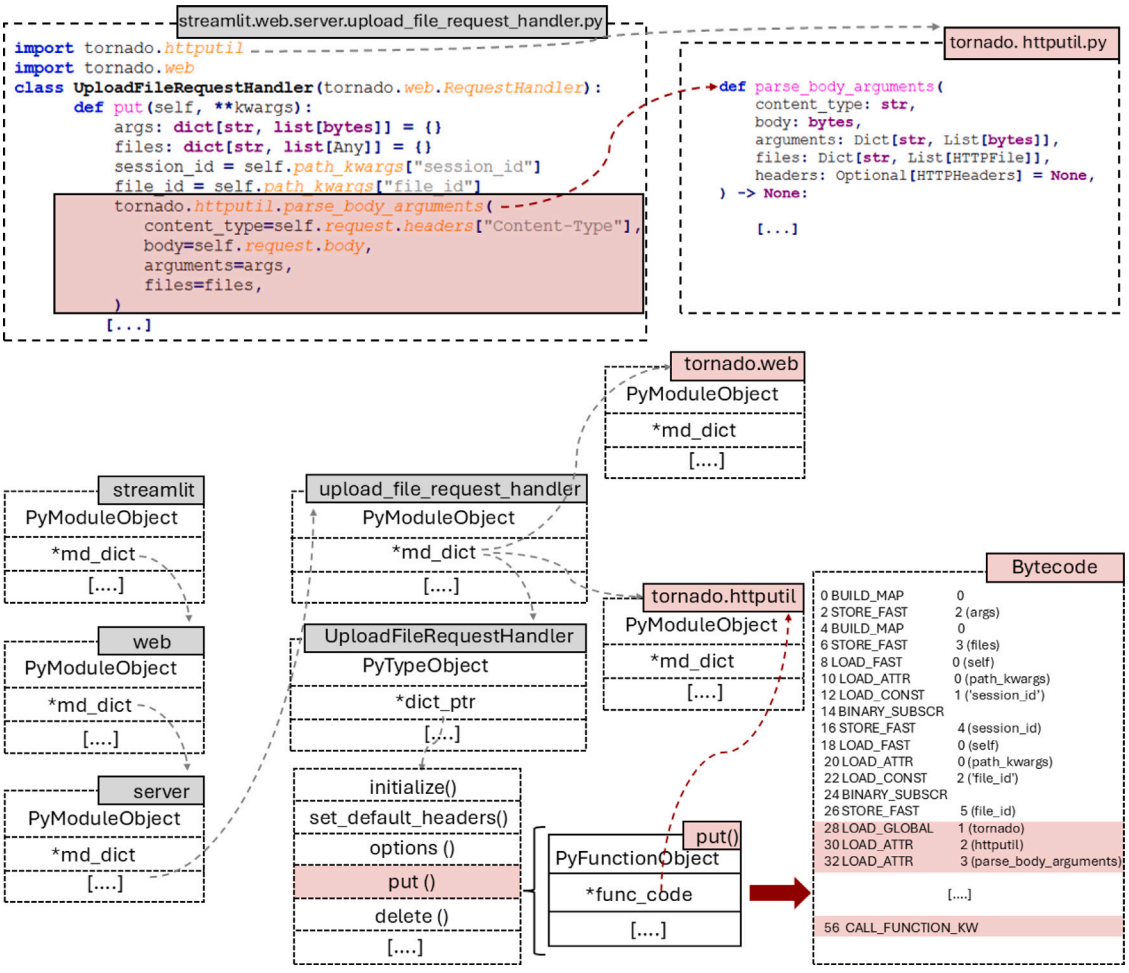


Fig. 4. Direct call path from *streamlit* to the vulnerable `tornado.httputil.parse_body_arguments()` function via the `put()` method, showing that the vulnerable function is reached.

Table 8
Function-level reachability analysis for the *tornado* vulnerabilities. Only *Streamlit* reaches a vulnerable function at runtime, while all other applications import only safe components.

Application	Tornado usage summary	Vulnerable module imported?	Vulnerable function reached?	Result
BentoML	`ioloop, gen, concurrent`	No	No	**Safe**
Flower	`web, ioloop, httpserver, escape, log`	No	No	**Safe**
Jet-Bridge	`ioloop, gen, web, iostream`	No	No	**Safe**
JupyterLab	`httputil, url_concat(), HTTPHeaders.parse_line()`	Yes	No	**Review**
Spyder	`ioloop, httpclient`	No	No	**Safe**
Streamlit	`httputil.parse_body_arguments()`	Yes	**Yes (CVE-2025-47287)**	**Vulnerable**

Summary: Package-level scanning flags 6/6 (100%), function-level scanning flags 1/6 (16.67%).

that merely include a vulnerable package from those that execute its vulnerable code paths, enabling security teams to prioritize remediation toward genuinely affected deployments. In incident-response scenarios, this capability further enables forensic analysts to determine from a memory dump whether a compromised dependency was actively reached by the application, providing evidence of actual exposure rather than structural presence alone.

Answer to RQ2

MEM-SBOM accurately identifies which applications reach vulnerable routines at runtime, revealing that only *Streamlit* invokes the affected *tornado* functions associated with *CVE-2025-47287*, demonstrating precise vulnerability reachability at runtime.

7.3.3. RQ3: Comparison with SBOM tools

This section first analyzes the limitations of existing SBOM tools in parsing and supporting heterogeneous metadata formats across the evaluated applications. It then compares the runtime package coverage, dependency-graph accuracy, and vulnerability detection effectiveness of tools capable of analyzing deployed environments, using MEM-SBOM as the runtime ground truth.

Metadata-based analysis. We collected applications from both *PyPI* and *GitHub* repositories. We observed that 30 *GitHub* projects have a `requirements.txt` file, a format supported by all tools, which explains the higher success rates observed in the *GitHub* dataset. The applications are grouped by metadata files into PEP 621-compliant projects (`pyproject.toml`), Poetry-managed projects (`pyproject.toml + poetry.lock`), and legacy or hybrid builds relying on `setup.py` and `setup.cfg`. The final group, `requirements.txt`

Table 9
Success rates (%) of SBOM tools across various metadata configurations for 51 Python applications (PyPI dataset).

Metadata configuration	#Apps	Syft	Trivy	CDXGen	CycloneDX-Python	SBOM4Python	ORT	Jake	PIP-SBOM	MEM-SBOM
pyproject.toml (PEP 621/setuptools)	8	0/8 (0%)	0/8 (0%)	0/8 (0%)	0/8 (0%)	8/8 (100%)	0/8 (0%)	0/8 (0%)	8/8 (100%)	8/8 (100%)
pyproject.toml (Poetry-managed)	4	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	4/4 (100%)	4/4 (100%)
setup.py ± setup.cfg	14	5/14 (35.71%)	0/14 (0%)	6/14 (42.86%)	0/14 (0%)	8/14 (57.14%)	6/14 (42.86%)	0/14 (0%)	13/14 (92.86%)	14/14 (100%)
pyproject.toml + setup.cfg	4	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	4/4 (100%)	0/4 (0%)	0/4 (0%)	4/4 (100%)	4/4 (100%)
pyproject.toml + setup.py ± setup.cfg	7	1/7 (14.29%)	0/7 (0%)	3/7 (42.86%)	0/7 (0%)	5/7 (71.43%)	2/7 (28.57%)	0/7 (0%)	5/7 (71.43%)	7/7 (100%)
requirements.txt ± (combined metadata)	14	10/14 (71.43%)	6/14 (42.86%)	10/14 (71.43%)	14/14 (100%)	14/14 (100%)	7/14 (50%)	13/14 (92.86%)	14/14 (100%)	14/14 (100%)

(combined metadata), denotes projects that include a `requirements.txt` file alongside other metadata files. Tables 9 and 10 highlight systematic weaknesses in metadata parsing across all tools. For PEP 621 projects, six tools (*Syft*, *Trivy*, *CDXGen*, *CycloneDX-Python*, *ORT*, and *Jake*) fail completely in both datasets, while only *PIP-SBOM* and *SBOM4Python* succeed (100%). In Poetry-managed projects, reliability depends almost entirely on the presence of a lock file. Without `poetry.lock`, all tools except *PIP-SBOM* fail to resolve dependencies; with it, most succeed except *SBOM4Python*, which lacks native support for `[tool.poetry.dependencies]`. *ORT* achieves partial success (66.7%) owing to an outdated Poetry runtime that cannot parse the `priority` field introduced in version 1.2.

For legacy `setup.py`–based projects, success rates vary widely due to the imperative and non-standard nature of dependency declarations. *Trivy*, *CycloneDX-Python*, and *Jake* provide no `setup.py` support (0%). Among regex-based tools, *Syft* captures only pinned dependencies (35.71% PyPI, 20% GitHub), while *CDXGen* performs slightly better (42.86%/80%) but fails on multiline or variable-defined lists and `extras_require` fields. *SBOM4Python* gives the highest coverage among pattern-matching tools (57.14%/60%) yet remains limited to single-line `install_requires`, producing empty SBOMs when absent (e.g., *Glances*). *ORT* achieves comparable rates (42.86%/80%) by executing `setup.py` and resolving dependencies through PyPI queries, a more flexible but nondeterministic strategy. Only *SBOM4Python* and *PIP-SBOM* correctly interpret auxiliary `setup.cfg` configurations, yielding complete outputs for hybrid declarative-imperative builds.

Although all evaluated tools support the `requirements.txt` file, they differ significantly in version handling and completeness. *Syft* and *Trivy* ignore unpinned dependencies (`==`), omitting over half of declared packages. *CDXGen* retains all constraints, while *SBOM4Python* and *ORT* include unpinned entries but assign versions as `none`, relying respectively on regex parsing and the `pip-requirements-parser` library (nexB, 2025). *Jake* and *CycloneDX-Python* exhibit inconsistent behavior due to API divergence; older parser versions in *Jake* omit unpinned entries, while newer releases used by *CycloneDX-Python* include them unconditionally. None of the tools correctly interpret pip-specific directives such as `-r` (recursive inclusion) or `-e` (editable installations), resulting in systematically incomplete SBOMs. Notably, MEM-SBOM is not affected by metadata configuration, producing precise runtime SBOMs that reflect actual execution.

These metadata-parsing limitations raise the question of whether SBOM tools can more accurately represent the deployed software state when analyzing installed environments rather than source metadata. To address this, we evaluate their performance in real runtime deployments. Notably, all subsequent runtime experiments are conducted on the PyPI-deployed versions of the applications.

Deployment-based analysis. We evaluated how *Syft*, *CycloneDX-Python*, *CDXGen*, and *PIP-SBOM* tools perform in real deployment environments, where dependencies are installed, version-resolved, and dynamically loaded. This analysis verifies whether these tools can accurately represent the runtime software state and produce accurate dependency graphs, using MEM-SBOM as the runtime ground truth. Given that these tools enumerate only direct dependencies, our comparison focuses on first-order dependency relationships, excluding transitive dependencies inferred through indirect imports.

As shown in Table 11, both *Syft* and *CycloneDX-Python* demonstrate stable behavior with high recall (94.81% and 90.9%) and near-perfect version accuracy (>99.9%) by directly inspecting installed distributions and parsing metadata from the `site-packages` directory of the active virtual environment. However, both tools report all installed packages regardless of whether they are imported at runtime, resulting in moderate precision (59.73% and 60.34%). Consequently, they construct realistic dependency graphs that closely reflect the deployed state, achieving the highest dependency accuracy, edge accuracy, and overall graph completeness.

In contrast, the predictive behavior of *CDXGen* and *PIP-SBOM* leads to the lowest performance across all metrics. Although *CDXGen* infers dependencies by creating a temporary virtual environment and installing dependencies using the `--install-deps` flag, it still relies on static metadata analysis to construct its SBOM rather than observing actual runtime imports. *PIP-SBOM* instead resolves constraints in metadata files by predicting what dependencies *would* be installed rather than those actually present. Although it achieves the highest precision among the four tools (72.49%) due to its conservative dependency resolution, this predictive strategy omits conditional or optional dependencies and frequently misreports versions when resolution assumptions diverge from the real deployment, resulting in substantially lower dependency accuracy ($D_{acc} < 50\%$) and incomplete graphs ($G_{complete} < 50\%$), consistent with its weaker runtime recall. The F1 scores confirm this trade-off. Although *PIP-SBOM* attains the highest precision, its limited recall yields a balance comparable to *Syft* and *CycloneDX-Python* (F_P of 69.39% versus 71.20% and 70.49%), while no tool exceeds 72%, reflecting the inherent gap between statically derived and runtime package sets.

MEM-SBOM, by directly enumerating the interpreter's loaded modules, eliminates these limitations and achieves complete recall and precision (100%) with 99.3% version accuracy due to inconsistencies between the declared package version at installation and the version embedded in the loaded code (see Table 5). However, all these tools are inherently static, missing packages that are dynamically loaded or generated at runtime.

Since *Grype* relies on the generated SBOMs, any inaccuracies in these SBOMs directly affect the reported vulnerability set, as illustrated in Table 12. *Syft* achieves full coverage ($C_V = 100\%$) equivalent to MEM-SBOM, while *CycloneDX-Python* provides near-complete detection

Table 10
Success rates (%) of SBOM tools across various metadata configurations for 51 Python applications (GitHub dataset).

Metadata configuration	#Apps	Syft	Trivy	CDXGen	CycloneDX-Python	SBOM4Python	ORT	Jake	PIP-SBOM	MEM-SBOM
pyproject.toml (PEP 621/setuptools)	4	0/4 (0%)	0/4 (0%)	0/4 (0%)	0/4 (0%)	4/4 (100%)	0/4 (0%)	0/4 (0%)	4/4 (100%)	4/4 (100%)
pyproject.toml (Poetry-managed)	3	3/3 (100%)	3/3 (100%)	3/3 (100%)	3/3 (100%)	0/3 (0%)	2/3 (66.67%)	3/3 (100%)	3/3 (100%)	3/3 (100%)
setup.py ± setup.cfg	5	1/5 (20%)	0/5 (0%)	4/5 (80%)	0/5 (0%)	3/5 (60%)	4/5 (80%)	0/5 (0%)	5/5 (100%)	5/5 (100%)
pyproject.toml + setup.cfg	2	0/2 (0%)	0/2 (0%)	0/2 (0%)	0/2 (0%)	1/2 (50%)	0/2 (0%)	0/2 (0%)	2/2 (100%)	2/2 (100%)
pyproject.toml + setup.py ± setup.cfg	7	0/7 (0%)	1/7 (14.29%)	2/7 (28.57%)	0/7 (0%)	5/7 (71.43%)	3/7 (42.86%)	0/7 (0%)	7/7 (100%)	7/7 (100%)
requirements.txt ± (combined metadata)	30	23/30 (76.67%)	17/30 (56.67%)	27/30 (90%)	29/30 (96.67%)	30/30 (100%)	18/30 (60%)	30/30 (100%)	25/30 (83.33%)	30/30 (100%)

Table 11
Performance of SBOM tools in deployment environments.

Tool	C_P (%)	P_P (%)	F_P (%)	M_P (%)	R_P (%)	V_{acc} (%)	D_{acc} (%)	E_{acc} (%)	$G_{complete}$ (%)
Syft	94.81	59.73	71.20	0.90	4.29	99.96	73.81	71.76	62.24
CDXGen	76.55	49.53	51.60	17.69	5.76	52.11	48.05	54.89	49.63
CycloneDX-Python	90.90	60.34	70.49	4.45	4.65	99.98	72.23	75.52	71.66
PIP-SBOM	73.42	72.49	69.39	21.96	4.61	65.22	27.90	34.15	20.09
MEM-SBOM	100.00	100.00	100.00	0.00	0.00	99.32	100.00	100.00	100.00

Table 12
The impact of SBOM tools on vulnerability detection.

Tool	C_V (%)	P_V (%)	F_V (%)	M_V (%)	R_V (%)
Syft	100.00	75.07	80.00	0.00	0.00
CDXGen	27.48	23.58	23.70	0.00	72.52
CycloneDX-Python	96.30	88.49	91.35	0.00	3.70
PIP-SBOM	37.37	33.80	34.67	0.00	62.63
MEM-SBOM	100.00	100.00	100.00	0.00	0.00

(96.3%), missing only 3.7% of runtime vulnerabilities. In contrast, *CDXGen* and *PIP-SBOM* exhibit coverage below 40% and high runtime-only ratios ($R_V > 60\%$), consistent with their incomplete runtime visibility. Notably, M_V is zero for all tools, indicating that vulnerabilities reported by both `pip audit` and MEM-SBOM are consistently detected; the missed vulnerabilities arise entirely from runtime-only packages. Finally, P_V decreases when tools report vulnerabilities associated with packages listed in metadata but never imported at runtime, thereby introducing false positives and reducing the overall F1 score (F_V). By restricting analysis to modules observed in memory, MEM-SBOM eliminates such false positives and achieves perfect precision, recall, and F1 ($P_V = C_V = F_V = 100\%$).

Answer to RQ3

Across heterogeneous metadata configurations and deployed environments, existing SBOM tools remain constrained by the metadata they parse, providing only partial visibility into the application runtime state. As a result, they generate incomplete SBOMs and dependency graphs, which in turn impact the accuracy of vulnerability detection.

7.3.4. Performance overhead

Table 13 in Appendix reports the execution time of each MEM-SBOM pipeline stage across 23 representative applications. The total analysis time ranges from 70 s for *RPyC* (2 packages, 217 submodules, single process) to 3107 s for *MLflow* (34 packages, 1469 submodules, six processes), indicating that the total analysis time correlates directly with application complexity in terms of module count and process concurrency. Dependency graph construction and heap scanning constitute the two dominant costs, with the former being the largest single stage in most applications, accounting for up to 71% of the total execution time. This overhead is attributable to the traversal of each module's namespace dictionary, the recursive inspection of class and function objects, and the disassembly of callable bytecode for import and call extraction, where address-based memoization ensures that each function, code object, and module dictionary is analyzed at most once. The cost scales with the number of submodules; for instance, *Apache Superset* (2859 submodules across two processes) requires 1477 s for graph construction, whereas *Glances* (294 submodules) completes in 38 s.

Heap scanning represents the second most significant cost and becomes dominant for applications with large heap footprints (e.g., 1588 s for *MLflow* and 887 s for *DevPi-Server*), as it performs an exhaustive 8-byte aligned scan over all heap regions to recover module objects absent from higher extraction layers. Interpreter registry and garbage collector traversal incur comparatively lower overhead, as both operate over structured pointer chains. PyPI resolution time is determined by the number of packages requiring external API queries to resolve import-to-distribution name mappings, with *Mayan-EDMS* exhibiting the highest latency (37 s), and SBOM serialization remains negligible across all applications. The residual difference between the total time and the sum of the reported stages corresponds to uninstrumented steps, primarily module grouping, classification, and version-attribute extraction, which together account for less than 6% of the total time. Multi-process applications incur proportionally higher overhead, as each process necessitates independent extraction across all layers.

In terms of asymptotic complexity, every pipeline stage is linear in the state it analyzes. Let P denote the number of application processes, n the module objects in memory, F the total number of local and global references across all live stack frames, G the GC-tracked objects, A and H the arena and heap region sizes, D the total number of entries across the namespace dictionaries (`__dict__`) of all analyzed modules, B the total number of bytecode instructions across all disassembled callables, and V_f and E_f the nodes and edges of the function-level call graph. Module extraction requires $O(n)$ for the interpreter registry (L1), $O(F)$ for inspecting the frame namespaces across thread execution states (L2), $O(G)$ for garbage collector traversal (L3), and $O(A)$ and $O(H)$ for

the fixed-stride arena and heap scans (L4–L5). The arena scan is absorbed into the heap term since arena regions reside within the mapped heap area ($A \leq H$); similarly, the frame references scanned in L2 are dominated by the resident-object bounds (G and H). Dependency graph construction is $O(D+B)$, since address-based memoization ensures each dictionary entry and bytecode instruction is processed at most once. Reachability analysis performs a backward traversal that visits each function and call edge at most once per vulnerability sink, i.e., $O(V_f + E_f)$ per sink. The overall cost is therefore $O(P \cdot (H+G+D+B))$, linear in memory size and total loaded code, consistent with the empirical scaling observed in Table 13.

8. Discussion

8.1. Generalizability

Although MEM-SBOM targets the Python runtime, its implementation handles structural changes across Python versions 3.6 through 3.14. The underlying methodology generalizes beyond Python, as memory forensics has been applied to recover runtime state from various interpreted and compiled language runtimes (Sylve et al., 2012; Case and Richard, 2016; Wang et al., 2022; Manna et al., 2022; Ali et al., 2026). However, each runtime has distinct memory management internals, object layouts, and code-loading semantics, requiring language-specific adaptation. This is consistent with findings that language-specific tooling is more effective than cross-language approaches (Stalnaker et al., 2024; Halbritter and Merli, 2024).

Accordingly, the approach presented in this paper, including multi-source module recovery, version resolution, dependency graph construction, and vulnerability reachability analysis, is transferable to other runtimes, while the implementation of each stage must be tailored to the target runtime's structures and the artifacts it preserves in memory.

8.2. Limitations and future work

This work focuses primarily on the Python runtime, excluding native C/C++ extensions that operate outside Python's managed memory. Such extensions introduce additional attack surfaces through language boundary transitions and type conversions (Scholtes et al., 2025). Extending MEM-SBOM to support polyglot correlation between Python objects and native components would enhance cross-language visibility. Beyond native extensions, the module-level detection on which MEM-SBOM relies requires a valid `PyModuleObject` with a correctly typed `ob_type` pointer; adversarial techniques that violate this assumption fall outside the current detection surface, as discussed in the RQ1 analysis. Extending detection to code-object-level analysis and type-chain validation could address these boundaries. MEM-SBOM currently analyzes only components resident in process memory at the time of acquisition. Therefore, attacks that occur solely during installation remain unobserved. Integrating installation-phase monitoring or provenance tracing could capture such transient artifacts produced during setup routines and extend visibility beyond the runtime state. Vulnerability assessment depends on Grype's aggregation of public databases (e.g., NVD, OSV, and GitHub Security Advisories), which limits detection to known CVEs. Future work may incorporate semantic pattern analysis and anomaly detection over memory-derived artifacts to identify previously unseen vulnerabilities. Finally, this work assumes that the Python runtime environment itself is not malicious or compromised. Prior work (Park et al., 2018) showed that memory-safety flaws in interpreters can corrupt bytecode while preserving valid metadata. Deep inspection of opcode sequences and Python-to-native wrapper transitions could further expose such tampering and injected behaviors.

9. Conclusion

In this paper, we introduced MEM-SBOM, the first memory forensics framework that generates accurate SBOMs directly from the runtime state of Python applications. By traversing the interpreter registries, thread states, garbage collector, arenas, and heap regions, the framework recovers all resident modules, even those concealed from any single extraction source. The extracted modules are then filtered and grouped by their parent packages to isolate the application packages, whose versions are determined through a multi-source resolution process. The resulting packages are then analyzed at the object level to construct dependency graphs and perform fine-grained reachability analysis. Evaluation across 51 real-world Python applications demonstrated that MEM-SBOM achieves 100% extraction accuracy under non-adversarial conditions and detects ten inconsistency cases between the declared package version at installation and the version embedded in the loaded code. In a detailed analysis of six *tornado*-dependent applications, MEM-SBOM identifies *Streamlit* as the only application that executes the vulnerable routines at runtime, eliminating 83.3% of package-level false positives. It also recovers all runtime-only dependencies, 4%–6% of which are consistently missed by existing tools, resulting in more complete dependency graphs and more accurate vulnerability correlation. These results show that memory-based SBOMs provide the runtime visibility essential for software supply chain security and for effective incident response, particularly in dynamic language ecosystems where metadata and filesystem artifacts fail to reveal the actual runtime state.

CRediT authorship contribution statement

Hala Ali: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Data curation, Conceptualization, Visualization. **Andrew Case:** Writing – review & editing, Validation, Software, Methodology, Investigation, Conceptualization. **Irfan Ahmed:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Claude (Anthropic) for minor language refinement and organization. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

See Tables 13–15.

Data availability

MEM-SBOM is implemented as a suite of open-source Volatility 3 plugins, publicly available at https://github.com/HalaAli198/MEM-SBOM.

Table 13
Execution time (in seconds) of each MEM-SBOM pipeline stage across the evaluated applications.

Application	No. process	Module extraction			PyPI Res.	Dep. graph	SBOM Gen.	Total
		Interp.	GC	Heap				
Ajenti-Panel	2	31.5470	24.8264	147.5635	9.8227	277.3461	0.0002	499.5221
Apache Superset	2	110.5898	193.7469	720.0206	34.1179	1477.2424	0.0004	2558.2464
APScheduler	1	7.4153	10.1302	41.6666	1.8801	100.8729	0.0001	168.3850
Beets	1	9.6317	14.9697	60.8725	4.1821	197.1928	0.0001	294.9833
Daphne	1	12.7650	17.1280	56.0714	4.4077	241.7003	0.0002	338.8738
Datasette	1	12.7961	19.2912	62.4387	5.2985	153.0578	0.0001	258.9725
DevPi-Server	1	24.7898	28.9122	886.9767	13.3967	426.5959	0.0002	1391.5724
Django	2	30.0436	37.6165	146.4955	1.0476	205.7147	0.0001	426.6493
Errbot	1	33.2870	24.4145	176.6188	6.3509	300.2526	0.0002	551.1488
Fsociety	1	13.0577	11.6807	41.9969	2.5083	121.9793	0.0001	196.7942
Glances	1	8.9285	10.2914	54.3347	1.6506	37.5888	0.0001	118.4148
iRedis	1	31.3666	20.1427	74.6170	2.9059	172.5831	0.0001	307.4606
JupyterLab	1	31.3113	37.6963	163.5030	18.1393	503.3062	0.0003	764.6193
Lektor	1	27.3767	20.2447	112.0958	7.3445	146.7142	0.0002	320.0858
Locust	1	39.3769	21.7633	81.1114	6.4831	205.0137	0.0001	361.6525
Mayan-EDMS	2	137.1658	261.1133	626.2574	37.4523	1171.8160	0.0009	2291.7456
MLflow	6	240.3986	192.4972	1588.3315	17.2219	1028.6755	0.0002	3106.8143
Modoboa	2	74.7097	90.1584	307.7056	17.8429	849.1255	0.0004	1349.9302
RPyC	1	5.1419	6.6145	28.8747	0.7891	24.9732	0.0001	70.2977
RQ	1	12.3161	11.9878	47.2932	1.2742	98.2222	0.0001	176.1784
Scapy	1	13.4066	20.8471	107.6911	0.2500	107.8159	0.0001	255.3066
Scrapy	1	23.8848	25.3307	94.5348	7.8199	301.6585	0.0002	460.7804
Trytond	1	10.6680	15.8946	68.1664	3.6557	158.4046	0.0001	262.4465

Table 14
Evaluation dataset of 51 Python applications spanning 23 categories.

Application category	Application name (Version)
Web frameworks	Django (v4.2.23), Flask (v3.0.3), FastAPI (v0.116.1)
Web-based environments	Jupyter Notebook (v7.4.5), JupyterLab (v4.3.8)
Task scheduling systems	Apache Airflow (v3.0.0), Celery (v5.5.3), APScheduler (v4.0.0a5), Prefect (v3.4.17)
Machine learning platforms	MLflow (v2.17.2), BentoML (v1.3.5), Seldon Core (v1.18.2), Rasa (v3.6.21), Gradio (v4.44.1)
Data visualization	Apache Superset (v2.1.3), Orange (v3.38.1)
ASGI servers	Daphne (v4.1.2), Uvicorn (v0.33.0), Hypercorn (v0.17.3)
Admin panels	Ajenti-Panel (v2.2.11), Flower (v2.0.1), Jet-Bridge (v1.12.1), Wooey (v0.13.3), Streamlit (v1.40.1)
CLI tools	iRedis (v1.15.2), LiteCLI (v1.13.2), MyCLI (v1.30.0)
Static site generators	Lektor (v3.3.12), MkDocs (v1.6.1), Pelican (v4.10.2), Nikola (v8.3.3)
Email servers	Modoboa (v2.3.6), Salmon (v3.3.0)
Development tools	Supervisor (v4.3.0), DevPi-Server (v6.15.0), Spyder (v6.0.8)
Database tools	SQLite-Web (v0.6.4), Datasette (v0.64.3)
RPC servers	RPyC (v6.0.2), Zerorpc (v0.6.3)
System monitoring	Glances (v3.4.0.3)
Penetration testing	Fsociety (v3.2.9)
Network tools	Mininet (v2.3.0.dev6), Scapy (v2.6.1)
Web scraping	Scrapy (v2.11.2)
Audio management	Beets (v2.2.0)
Chatbots	Errbot (v6.2.0)
Business software	Trytond (v6.6.1)
Document management	Mayan-EDMS (v4.9.2)
Task queues	RQ (v2.3.3)
Testing tools	Locust (v2.25.0)

Table 15
Detected vulnerable packages across the evaluated Python applications.

Project	Version	Affected package	Package version	Critical	High	Medium	Low
Ajenti-Panel	2.2.11	urllib3	2.2.3	–	–	2	–
BentoML	1.3.5	bentoml	1.3.5	3	2	1	–
		tornado	6.4.2	–	1	–	–
		starlette	0.44.0	–	–	1	–
DevPi-Server	6.15.0	waitress	3.0.0	1	1	–	–
		urllib3	2.2.3	–	–	2	–
Django	4.2.23	django	4.2.23	–	1	–	–
Errbot	6.2.0	cryptography	41.0.7	–	2	2	–
		jinja2	3.1.2	–	–	5	–
		requests	2.31.0	–	–	2	–
FastAPI	0.116.1	starlette	0.44.0	–	–	1	–
		urllib3	2.2.3	–	–	2	–
Flower	2.0.1	tornado	6.4.2	–	1	–	–
Fsociety	3.2.9	urllib3	2.2.3	–	–	2	–
Gradio	4.44.1	gradio	4.44.1	1	7	7	–
		starlette	0.44.0	–	–	1	–
		urllib3	2.2.3	–	–	2	–
Hypercorn	0.17.3	h2	4.1.0	–	–	1	–
Jet-Bridge	1.12.1	tornado	5.1.1	–	2	4	–
		pymongo	4.1.1	–	–	1	–
		urllib3	2.2.3	–	–	2	–
JupyterLab	4.3.8	tornado	6.4.2	–	1	–	–
		urllib3	2.2.3	–	–	2	–
Jupyter Notebook	7.4.5	jupyterlab	4.4.5	–	–	–	1
Lektor	3.3.12	werkzeug	2.3.8	–	1	2	–
		urllib3	2.2.3	–	–	2	–
Locust	2.25.0	flask_cors	5.0.0	–	–	3	–
		urllib3	2.2.3	–	–	2	–
Mayan-EDMS	4.9.2	django	4.2.23	–	1	–	–
		pypdf	5.1.0	–	–	1	–
		urllib3	1.26.20	–	–	1	–
MLflow	2.17.2	mlflow	2.17.2	–	–	3	–
		urllib3	2.2.3	–	–	2	–
MyCLI	1.30.0	sqlparse	0.4.4	–	1	–	–
		paramiko	2.11.0	–	–	1	–
Nikola	8.3.3	urllib3	2.2.3	–	–	2	–
Rasa	3.6.21	keras	2.12.0	1	1	1	–
		tensorflow	2.12.0	–	1	–	–
		future	1.0.0	–	1	–	–
		aiohttp	3.9.5	–	–	1	–
		pymongo	4.3.3	–	–	1	–
		urllib3	1.26.20	–	–	1	–
Scrapy	2.11.2	urllib3	2.2.3	–	–	2	–
Seldon Core	1.18.2	flask_cors	3.0.10	–	1	4	–
		gunicorn	20.1.0	–	2	–	–
		werkzeug	2.2.3	–	1	3	–
Spyder	6.0.8	tornado	6.4.2	–	1	–	–
		urllib3	2.2.3	–	–	2	–
		aiohttp	3.10.11	–	–	–	1
Streamlit	1.40.1	tornado	6.4.2	–	1	–	–
		urllib3	2.2.3	–	–	2	–
Apache Superset	2.1.3	pyarrow	10.0.1	1	–	–	–
		cryptography	39.0.2	–	2	3	3
		sqlparse	0.4.4	–	1	–	–
		werkzeug	2.3.3	–	1	3	–
		flask_caching	–	–	–	1	–
		urllib3	2.2.3	–	–	2	–
Zerorpc	0.6.3	future	1.0.0	–	1	–	–

References

Ali, H., Case, A., Ahmed, I., 2025a. Leveraging memory forensics to investigate and detect illegal 3D printing activities. Forensic Sci. Int.: Digit. Investig. 53, 301925. http://dx.doi.org/10.1016/j.fsidi.2025.301925.

Ali, H., Case, A., Ahmed, I., 2025b. Memory analysis of the python runtime environment. Forensic Sci. Int.: Digit. Investig. 53, 301920. http://dx.doi.org/10.1016/j.fsidi.2025.301920.

Ali, H., Case, A., Ahmed, I., 2026. Memory Forensics Techniques for Automated Detection and Analysis of Go Malware. Forensic Science International: Digital Investigation 57, 302118.

Anchore, 2025a. Grype: Vulnerability scanner for container images and filesystems. https://github.com/anchore/grype. GitHub repository. (Accessed 15 November 2025).

Anchore, 2025b. Syft: A CLI tool and library for generating SBOMs. https://github.com/anchore/syft. GitHub repository. (Accessed 11 March 2025).

Aqua Security, 2025. Trivy: A comprehensive and versatile security scanner. https://github.com/aquasecurity/trivy. GitHub repository. (Accessed 11 March 2025).

Benedetti, G., Cofano, S., Brighente, A., Conti, M., 2025. The impact of SBOM generators on vulnerability assessment in python: A comparison and a novel approach. In: International Conference on Applied Cryptography and Network Security. Springer, pp. 487–509. http://dx.doi.org/10.1007/978-3-031-95764-2_19.

Beyer, B., Jones, C., Petoff, J., Murphy, N.R., 2016. Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, Inc.

Bi, T., Xia, B., Xing, Z., Lu, Q., Zhu, L., 2024. On the way to SBOM: Investigating design issues and solutions in practice. ACM Trans. Softw. Eng. Methodol. 33 (6), 1–25. http://dx.doi.org/10.1145/3654442.

Bidart, N., 2025. Django security releases issued: 5.2.8, 5.1.14, and 4.2.26. https://www.djangoproject.com/weblog/2025/nov/05/security-releases/. (Accessed 11 November 2025).

Bufalino, J., Di Francesco, M., Blaise, A., Secci, S., 2025. SBOMproof: Beyond alleged SBOM compliance for supply chain security of container images. http://dx.doi.org/10.48550/arXiv.2510.05798, arXiv preprint arXiv:2510.05798. Preprint.

Case, A., Richard, III, G.G., 2016. Detecting Objective-C malware through memory forensics. Digit. Investig. 18, S3–S10.

Chen, V., 2025. Awesome python. https://github.com/vinta/awesome-python. GitHub repository. (Accessed 02 October 2025).

Cofano, S., Benedetti, G., Dell'Amico, M., 2024. SBOM generation tools in the python ecosystem: An in-detail analysis. In: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications. TrustCom, IEEE, pp. 427–434. http://dx.doi.org/10.1109/TrustCom63139.2024.00077.

Cybersecurity and Infrastructure Security Agency (CISA), 2021. Defending Against Software Supply Chain Attacks. Technical Report, U.S. Department of Homeland Security, https://www.cisa.gov/sites/default/files/publications/defending_against_software_supply_chain_attacks_508.pdf. TLP:WHITE. (Accessed 15 February 2025).

Cybersecurity and Infrastructure Security Agency (CISA), 2023. Types of software bill of material (SBOM) documents. https://www.cisa.gov/sites/default/files/2023-04/sbom-types-document-508c.pdf. (Accessed 11 March 2025).

Cybersecurity and Infrastructure Security Agency (CISA), 2025. 2025 Minimum Elements for a Software Bill of Materials (SBOM): Public Comment Draft. Technical Report, U.S. Department of Homeland Security, https://www.cisa.gov/sites/default/files/2025-08/2025_CISA_SBOM_Minimum_Elements.pdf. Draft for public comment.

CycloneDX Project, 2025a. Cdxgen: Universal polyglot SBOM generator. https://github.com/CycloneDX/cdxgen. GitHub repository. (Accessed 26 April 2025).

CycloneDX Project, 2025b. CycloneDX: A standard for software bill of materials (SBOM). https://cyclonedx.org/. (Accessed 12 March 2025).

CycloneDX Project, 2025c. Cyclonedx-python: SBOM generator for python projects. https://github.com/CycloneDX/cyclonedx-python. GitHub repository. (Accessed 26 April 2025).

Dalia, G., Visaggio, C.A., Di Sorbo, A., Canfora, G., 2024. SBOM ouverture: What we need and what we have. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. Association for Computing Machinery, pp. 1–9. http://dx.doi.org/10.1145/3664476.3669975.

Eclipse Foundation, 2022. Eclipse jbom: Runtime and static SBOMs for Java applications. https://projects.eclipse.org/projects/technology.jbom. (Accessed 25 June 2026).

Fiedler, M., 2023. Inbound Malware volume report. https://blog.pypi.org/posts/2023-09-18-inbound-malware-reporting/. (Accessed 10 November 2025).

Golubin, A., 2019. Memory management in python. https://rushter.com/blog/python-memory-managment/. Last updated July 8, 2019. (Accessed 15 November 2025).

Halbritter, A., Merli, D., 2024. Accuracy evaluation of SBOM tools for web applications and system-level software. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. Association for Computing Machinery, pp. 1–9. http://dx.doi.org/10.1145/3664476.3670926.

Harrison, A., 2025. SBOM4python: Open-source tool to generate an SBOM for installed python modules. https://github.com/anthonyharrison/sbom4python. GitHub repository. (Accessed 26 April 2025).

Kampourakis, V., Kavallieratios, G., Gkioulos, V., Katsikas, S., 2025. Cracks in the chain: A technical analysis of real-life supply chain security incidents. Comput. Secur. 159, 104673. http://dx.doi.org/10.1016/j.cose.2025.104673.

Kawaguchi, N., Hart, C., 2024. On the deployment control and runtime monitoring of containers based on consumer side SBOMs. In: 2024 IEEE 21st Consumer Communications & Networking Conference. CCNC, IEEE, pp. 1022–1025. http://dx.doi.org/10.1109/CCNC51664.2024.10454654.

Lakshmanan, R., 2025. Malicious python packages on PyPI downloaded 39,000+ times, steal sensitive data. https://thehackernews.com/2025/04/malicious-python-packages-on-pypi.html. (Accessed 10 November 2025).

Manna, M., Case, A., Ali-Gombe, A., Richard, III, G.G., 2022. Memory analysis of .NET and .NET core applications. Forensic Sci. Int.: Digit. Investig. 42, 301404. http://dx.doi.org/10.1016/j.fsidi.2022.301404.

Mirakhorli, M., Garcia, D., Dillon, S., Laporte, K., Morrison, M., Lu, H., Koscinski, V., Enoch, C., 2024. A landscape study of open source and proprietary tools for software bill of materials (SBOM). http://dx.doi.org/10.48550/arXiv.2402.11151, arXiv preprint arXiv:2402.11151. Preprint.

Nahum, M., Grolman, E., Maimon, I., Mimran, D., Brodt, O., Elyashar, A., Elovici, Y., Shabtai, A., 2024. OSSIntegrity: Collaborative open-source code integrity verification. Comput. Secur. 144, 103977. http://dx.doi.org/10.1016/j.cose.2024.103977.

National Counterintelligence and Security Center (NCSC), 2021. Software Supply Chain Attacks. Technical Report, Office of the Director of National Intelligence (ODNI), https://www.dni.gov/files/NCSC/documents/supplychain/Software-Supply-Chain-Attacks.pdf. (Accessed 15 November 2025).

Nelson, N., 2023. Novel PyPI malware uses compiled python bytecode to evade detection. https://www.darkreading.com/application-security/novel-pypi-malware-compiled-python-bytecode-evade-detection. (Accessed 06 January 2025).

nexB, 2025. Pip-requirements-parser. https://github.com/nexB/pip-requirements-parser. GitHub repository. (Accessed 05 October 2025).

Office of the Director of National Intelligence (ODNI), National Security Agency (NSA), Cybersecurity and Infrastructure Security Agency (CISA), 2023. Securing the software supply chain: Recommended practices for software bill of materials consumption. https://www.cisa.gov/sites/default/files/2024-08/SECURING_THE_SOFTWARE_SUPPLY_CHAIN_RECOMMENDED_PRACTICES_FOR_SOFTWARE_BILL_OF_MATERIALS_CONSUMPTION-508.pdf. (Accessed 11 March 2025).

Oligo Security, 2023. Scaling runtime security: How eBPF is solving decade-long challenges. https://www.oligo.security/blog/scaling-runtime-security-how-ebpf-is-solving-decade-long-challenges. (Accessed 25 June 2026).

OSS Review Toolkit, 2025. OSS review toolkit (ORT): FOSS policy automation and SBOM generation framework. https://github.com/oss-review-toolkit/ort. GitHub repository. (Accessed 27 April 2025).

Park, T., Lettner, J., Na, Y., Volckaert, S., Franz, M., 2018. Bytecode corruption attacks are real—and how to defend against them. In: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Springer, pp. 326–348. http://dx.doi.org/10.1007/978-3-319-93411-2_15.

Python Software Foundation, 2024. Python documentation: Site — Site-specific configuration hook. https://docs.python.org/3/library/site.html. (Accessed 09 January 2025).

Reichelt, D.G., Bulej, L., Jung, R., Van Hoorn, A., 2024. Overhead comparison of instrumentation frameworks. In: Companion of the 15th ACM/SPEC International Conference on Performance Engineering. pp. 249–256. http://dx.doi.org/10.1145/3629527.3652269.

Rezilion, 2022. Rezilion adds dynamic software bill of materials to its platform. https://www.prnewswire.com/news-releases/rezilion-adds-dynamic-software-bill-of-materials-to-its-platform-providing-organizations-with-continuous-view-of-all-software-components-and-exploitable-risk-301548653.html. (Accessed 25 June 2026).

Samaana, H., Costa, D.E., Shihab, E., Abdellatif, A., 2025. A machine learning-based approach for detecting malicious PyPI packages. In: Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing. pp. 1617–1626. http://dx.doi.org/10.1145/3672608.3707756.

Scholtes, R., Khodayari, S., Staicu, C.-A., Pellegrino, G., 2025. CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions. In: 2025 IEEE 10th European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 153–168. http://dx.doi.org/10.1109/EuroSP63326.2025.00018.

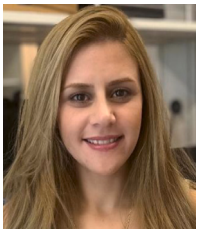
Sharma, A., Wittlinger, M., Baudry, B., Monperrus, M., 2024. SBOM.EXE: Countering dynamic code injection based on software bill of materials in java. http://dx.doi.org/10.48550/arXiv.2407.00246, arXiv preprint arXiv:2407.00246. Preprint.

Smith, E.V., 2012. PEP 420: Implicit namespace packages. https://peps.python.org/pep-0420/. (Accessed 30 October 2025).

Snow, E., 2013. PEP 451: A ModuleSpec type for the import system. https://peps.python.org/pep-0451/. (Accessed 30 October 2025).

Software Package Data Exchange (SPDX), 2025. SPDX: Open standard for software bill of materials (SBOM). https://spdx.dev/. (Accessed 12 March 2025).

Sonatype, 2025. 10Th annual state of the software supply chain. https://www.sonatype.com/state-of-the-software-supply-chain/introduction. (Accessed 10 March 2025).
Sonatype Nexus Community, 2025. Jake: Dependency analysis and SBOM generation tool. https://github.com/sonatype-nexus-community/jake/tree/803ec4f63e2c117352463065cf69d12a69f38450. GitHub repository. (Accessed 02 October 2025).
Stalnaker, T., Wintersgill, N., Chaparro, O., Di Penta, M., German, D.M., Poshyvanyk, D., 2024. Boms away! Inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. Association for Computing Machinery, pp. 1–13. http://dx.doi.org/10.1145/3597503.3623347.
Stufft, D., 2015. PEP 503: Simple Repository API. https://peps.python.org/pep-0503/. (Accessed 12 November 2025).
Sylve, J., Case, A., Marziale, L., Richard, G.G., 2012. Acquisition and analysis of volatile memory from Android devices. Digit. Investig. 8 (3–4), 175–184. http://dx.doi.org/10.1016/j.diin.2011.10.003.
Tooling and Implementation Working Group, 2024. Framing software component transparency: Establishing a common software bill of materials (SBOM). Third Edition. https://www.cisa.gov/sites/default/files/2024-10/SBOM%20Framing%20Software%20Component%20Transparency%202024.pdf.
Volatility Foundation, 2025. Volatility 3: The volatile memory extraction framework. https://github.com/volatilityfoundation/volatility3. GitHub repository. (Accessed 01 January 2025).
Vu, D.-L., Newman, Z., Meyers, J.S., 2023. Bad snakes: Understanding and improving python package index malware scanning. In: Proceedings of the 45th International Conference on Software Engineering. ICSE, IEEE, pp. 499–511. http://dx.doi.org/10.1109/ICSE48619.2023.00052.
Wang, J., 2024. Malicious PyPI package zlibxjson targets browser data and credentials. https://www.fortinet.com/blog/threat-research/malicious-pypi-package-zlibxjson-steals-browser-data. (Accessed 10 November 2025).
Wang, E., Zurowski, S., Duffy, O., Thomas, T., Baggili, I., 2022. Juicing V8: A primary account for the memory forensics of the V8 JavaScript engine. Forensic Sci. Int.: Digit. Investig. 42, 301400. http://dx.doi.org/10.1016/j.fsidi.2022.301400.
Xia, B., Bi, T., Xing, Z., Lu, Q., Zhu, L., 2023. An empirical study on software bill of materials: Where we stand and the road ahead. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 2630–2642. http://dx.doi.org/10.1109/ICSE48619.2023.00219.
Yu, S., Song, W., Hu, X., Yin, H., 2024. On the correctness of metadata-based SBOM generation: A differential analysis approach. In: 2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 29–36. http://dx.doi.org/10.1109/DSN58291.2024.00018.
Zahan, N., Lin, E., Tamanna, M., Enck, W., Williams, L., 2023. Software bills of materials are required. Are we there yet? IEEE Secur. Priv. 21 (2), 82–88. http://dx.doi.org/10.1109/MSEC.2023.3237100.
Zhao, Y., Zhang, Y., Chacko, D., Cappos, J., 2024. Covsbom: Enhancing software bill of materials with integrated code coverage analysis. In: 2024 IEEE 35th International Symposium on Software Reliability Engineering. ISSRE, IEEE, pp. 228–237. http://dx.doi.org/10.1109/ISSRE62328.2024.00031.



Hala Ali received her Ph.D. in Computer Science from Virginia Commonwealth University (VCU), Richmond, Virginia, USA, in 2026. Her research interests include memory forensics, malware analysis, and software supply chain security. She received her bachelor's degree in Informatics Engineering from AlBaath University, Homs, Syria, in 2016, and her master's degree in Computer Science and Information Security from the National Institute of Technology, Warangal, Telangana, India, in 2020. She is a recipient of the Best Paper Award at the 25th and 26th Annual Digital Forensics Research Conference (DFRWS USA 2025 and 2026). Her work has been presented at venues including DFRWS, WiCyS, PyCon US, BSides Las Vegas, and Black Hat USA Arsenal.

Andrew Case is the Director of Research at Volexity and has significant experience in incident response handling, digital forensics, and malware analysis. He is a core developer of Volatility, the most widely used open-source memory forensics framework, and a co-author of the highly popular and technical forensics analysis book "The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory". Case received his master's degree in Computer Science from the University of New Orleans (UNO), New Orleans, Louisiana, USA. He has presented at major industry conferences, including Black Hat, DEF CON, RSA, SecTor, DFRWS, and OMFW.

Irfan Ahmed is a Professor of Computer Science at Virginia Commonwealth University (VCU) , Richmond, Virginia, USA . He is the Director of the Security and Forensics Engineering (SAFE) Research Lab and a faculty fellow of VCU Cybersecurity Center. Before joining VCU, Ahmed was a Canizaro-Livingston Endowed Assistant Professor in Cybersecurity at the University of New Orleans (UNO), New Orleans, Louisiana, USA. His research interests are broadly in cybersecurity, currently focusing on digital forensics, malware, and cyber physical system security, including industrial control systems, and additive manufacturing. Ahmed is a recipient of the ORAU Ralph E. Powe Junior Faculty Enhancement Award, the Outstanding Research Award from the American Academy of Forensic Sciences (AAFS), and the UNO's Early Career Research Prize, the Dean's Faculty Fellow Award at the VCU College of Engineering, USCYBERCOM Commander Award, and USCYBERCOM Guardian Award.